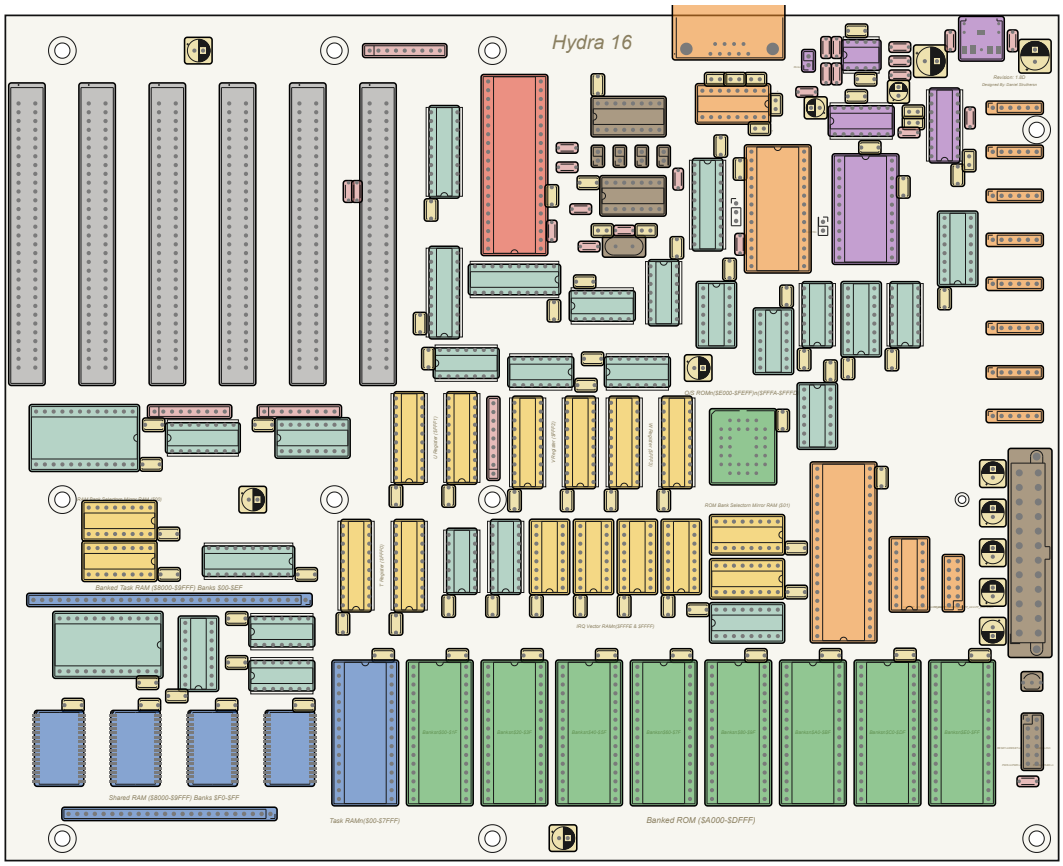
THE HYDRA PROJECT

# HYDRA-16

## *Educator's Course Guide*




Course plans, lesson notes, safety, assessment and answers
for the User's and Builder's Manual and the Student Companion

**HydraOS 1.0**

# About This Guide

This guide is for whoever is leading other people through the Hydra-16: a teacher with a class, a professor with a lab section, a club leader, or a parent with one determined child. It doesn't teach the Hydra. The *User's and Builder's Manual* does that. This guide is about teaching *with* it: what to cover, in what order, how long it takes, what goes wrong, how to keep a room safe, and how to tell whether anyone learned anything.

## The three books

| BOOK | WHO HOLDS IT | WHAT IT'S FOR |
|---|---|---|
| **The User's and Builder's Manual** | Everyone | The tutorial and the reference. Every fact is in here, and only here. |
| **The Student Companion** | Each student | A workbook: the build log, 26 lab sheets, 7 projects, reference cards, a logbook. It has questions, and no answers. |
| **The Educator's Course Guide** | You | Course plans, notes on each lesson, safety, assessment, and the answers. |

Wherever this guide says **Chapter 6** or **section 6.4**, it means the manual's. **Lab 3**, **Project 2**, **Unit 4** and **Card 5** are the Companion's. This guide's own chapters are called **Topics**.

## What you need to know first

Less than you might fear. The manual was written so that a twelve-year-old can follow it alone, and you can keep a chapter ahead of a class. But do these three things before the first lesson:

1. **Do Labs 1 to 3 yourself**, on the same computers the students will use. An hour and a half, and you'll have met every snag they will.
2. **Read the manual's Chapter 6** properly. It's the one chapter everything else stands on.
3. **Decide how the Hydra will be there** in the room: emulated, built already, or built by the class. Topic 3 compares them. It's the decision that shapes everything else.

If you're going to build, also solder something yourself, and read Topic 4 and Topic 6 before you order anything.

## What this guide won't tell you

**What a kit costs, and where to get one.** That changes, and it isn't in the Hydra's repository. Topic 3 lists what a kit is made of, so that you can price it; the manual's Appendix I is the full parts list.

**Which standards your course meets.** Those are yours to map. Topic 1 lists what the Hydra teaches, grouped the way most computing curricula group things, to make the mapping quick.

## How the answers were made

Every transcript in this guide's answers was typed into a Hydra, the same one the manual used (three RAM modules, the sample SD card, no Vera X, no clock chip), and what's printed is what it said. When your students' Hydras differ from that one, their answers will differ in the same small ways that the manual's section "This book's Hydra" lists. A different answer isn't always a wrong one: check the *reasoning*.

> **NOTE:** Keep this guide out of the students' hands, and not only because of the answers. Half of what a lab teaches is the frustration that comes just before the answer.

# Contents


**About This Guide** ..... 2

## PART ONE: PLANNING A COURSE

**1 What the Hydra Teaches** ..... 8
- The big ideas ..... 8
- What students will be able to do ..... 9
- Where it fits ..... 9
- Who it's for ..... 10
- What students need first ..... 10

**2 Course Shapes** ..... 11
- The pieces ..... 11
- Shape A: a week (emulator only) ..... 13
- Shape B: a nine-week unit, "Inside a Computer" ..... 13
- Shape C: a semester, "From Chips to Programs" ..... 13
- Shape D: a year, with a build ..... 14
- Shape E: a club ..... 14
- Shape F: a university laboratory ..... 15
- Choosing ..... 15

**3 The Room, the Kit and the Emulator** ..... 17
- The emulator ..... 17
- The kit ..... 18
- A bench for soldering ..... 19
- The room for keyboard lessons ..... 20

**4 Safety** ..... 21
- The hazards, and what to do about each ..... 21
- The rules, for the wall ..... 22
- Before the first session ..... 22
- How many at once ..... 23
- When something happens ..... 23
- The quieter risks ..... 23

## PART TWO: TEACHING IT

**5 Lesson by Lesson** ..... 25
- The shape of a lab lesson ..... 25
- Labs 1 to 3: meeting the Hydra ..... 26
- Labs 4 to 9: inside HydraOS ..... 27
- Labs 10 to 14: devices ..... 29
- Labs 15 to 23: programming languages ..... 30
- Labs 24 to 26: machine language ..... 33

**6 Running a Build** ..... 35
- Is the class ready? ..... 35

Sharing one board 35
The order of work, with a class 36
The surface-mount chips 37
The first switch-on 37
Fault-finding as a lesson 37
What the students keep 38
A build without a class 38

**7 Assessment 39**
What to assess, and what not to 39
Lab sheets 39
The Skills Passport 40
Projects 40
The build 41
A written test 41
Reporting 42

**8 Reaching Everyone 43**
The student who races ahead 43
The student who is struggling 43
Younger students (12 and 13) 44
Older students and adults 44
Students who can't solder 45
Seeing, hearing and moving 45
Language 45
Girls, and everyone else who's been told this isn't for them 46

**9 When Things Go Wrong in Class 47**
The emulator 47
At the prompt 48
A real Hydra 49
The lesson itself 49

## PART THREE: THE ANSWERS

**10 The Build Log 51**
Unit 1. Before You Build 51
Unit 2. Soldering the Main Board 52
Unit 3. Power, ROMs and Chips 52
Unit 4. The Memory Card 52
Unit 5. First Switch-On 53

**11 Labs 1 to 9 54**
Lab 1. Meet the Computer 54
Lab 2. A Hydra on Your PC 55
Lab 3. First Steps 56
Lab 4. From Power to Prompt 58
Lab 5. Talking to the Shell 60
Lab 6. Scripts 62
Lab 7. The Tree of Files 64
Lab 8. Windows 66
Lab 9. A Command Hunt 66

**12 Labs 10 to 15** ..... **68**
Lab 10. Devices Are Files ..... 68
Lab 11. Disks ..... 69
Lab 12. Sound ..... 72
Lab 13. Pictures ..... 73
Lab 14. Pins, and the System's Own Files ..... 74
Lab 15. Numbers and Bases ..... 76

**13 Labs 16 to 23** ..... **78**
Lab 16. HyForth: The Stack ..... 78
Lab 17. HyForth: Decisions, Loops and Programs ..... 80
Lab 18. hylang: Expressions and Functions ..... 81
Lab 19. hylang: Lists, Decisions and Programs ..... 84
Lab 20. HyBASIC: First Programs ..... 86
Lab 21. HyBASIC: Bigger Programs ..... 88
Lab 22. Your Language Meets the Hydra ..... 90
Lab 23. Three Languages ..... 92

**14 Labs 24 to 26** ..... **94**
Lab 24. The 65C02, on Paper ..... 94
Lab 25. Your First Machine Code ..... 96
Lab 26. Taking Programs Apart ..... 98

**15 The Projects** ..... **103**
Project 1. A Quiz ..... 103
Project 2. Turtle Pictures ..... 104
Project 3. A Round for Two Voices ..... 104
Project 4. Your Own Command ..... 105
Project 5. A Program in the Background ..... 106
Project 6. A Traffic Light ..... 107
Project 7. Faster, with Machine Code ..... 107

**16 A Bank of Questions** ..... **109**
Set 1. The machine (Chapters 1 and 7; Labs 1 and 4) ..... 109
Set 2. Using HydraOS (Chapters 6 and 8 to 11; Labs 3 and 5 to 9) ..... 110
Set 3. Devices (Chapters 12 to 17; Labs 10 to 14) ..... 110
Set 4. Numbers and programming (Chapters 18 to 25; Labs 15 to 23) ..... 111
Set 5. Machine language (Chapters 26 to 29; Labs 24 to 26) ..... 112
Set 6. Building (Chapters 3 to 5; the build log) ..... 112



PART ONE

# Planning a Course

What the Hydra is good for teaching, the shapes a course can take, what the room needs, and how to keep it safe. Read this part before you promise anybody anything.


- Topic 1. What the Hydra Teaches
- Topic 2. Course Shapes
- Topic 3. The Room, the Kit and the Emulator
- Topic 4. Safety

TOPIC 1

# What the Hydra Teaches

Most school computing starts at the top: an app, a web page, a block of Python. The machine underneath stays a mystery, and so does the reason anything works. The Hydra starts at the bottom. A student who has soldered the memory chips in, watched the self test count them, and then asked `free` how much there is, *knows* what memory is, in a way no diagram can give.

And the Hydra is small enough to know completely. One processor with six registers. One board whose every chip has a job you can name. An operating system whose start-up fits in eight steps. That completeness is the point: it's a whole computer that one person can hold in their head.

## The big ideas

Everything in the three books serves one of these seven ideas. They're the answer to "what will my students actually know?"

| THE IDEA | WHERE THE MANUAL TEACHES IT | THE LABS THAT MAKE IT REAL |
|---|---|---|
| **A computer is a machine that follows instructions**, made of a few kinds of part: a processor, memory, and ways in and out | Chapters 1, 26 | 1, 24 |
| **Everything is numbers**, and the same number can be written in many ways | Sections 1.6, 26.2; Chapter 18 | 1, 15, 24 |
| **An operating system shares one machine among many programs** | Chapters 7, 10 | 4, 8; Project 5 |
| **Naming things, and one tree for everything**: files, devices, even running programs | Chapters 9, 12, 17 | 7, 10, 14 |
| **Small tools join up**: a program does one thing, and pipes and scripts combine them | Chapters 8, 11 | 5, 6, 9; Project 4 |
| **A language is a way of thinking**: the same problem looks different in each | Chapters 19 to 25 | 16 to 23 |
| **Layers**: each level of a computer is built from the one below, and you can open any of them | Chapters 26 to 29 | 24 to 26; Project 7 |

And under all seven runs an eighth, which is not about computers: **when it doesn't work, there's a reason, and you can find it**, by observing carefully, changing one thing at a time, and writing down what happened. The build log and the fault diary exist to teach that. It may be the most valuable thing a student takes away.

## What students will be able to do

By the end of the whole course (few courses will do it all: Topic 2):

- **Build**: solder a board of 2,200 joints to a working standard; read component values; test for power faults; find and fix a fault methodically.
- **Operate**: use a command line fluently; manage files; run several programs at once; automate a job with a script.
- **Program**: write, test and debug programs of 20 to 50 lines in at least one of three languages; read simple programs in the other two; choose a language for a job, with reasons.
- **Explain**: how a processor runs a program; how memory is addressed; how one processor runs many tasks; what an operating system, a driver, a file system and a shell are for.
- **Go below**: follow machine code by hand; write, assemble, disassemble and single-step a small program.

## Where it fits

For mapping to a curriculum, here's what the Hydra covers under the headings most computing frameworks use.

| STRAND | WHAT THE HYDRA OFFERS | WHERE |
|---|---|---|
| **Computing systems** (hardware, and how hardware and software work together) | Its strongest suit: every component named and handled; memory, the bus, the processor; drivers; fault finding | Chapters 1, 3 to 7, 12 to 17, 26 |
| **Data and its representation** | Binary and hex; character codes; exact and inexact numbers; files as bytes | Sections 1.6, 26.2; Chapter 18; Appendix E; Lab 9's hex dump |
| **Algorithms and programming** | Sequence, selection, iteration, variables, procedures, recursion, decomposition, testing and debugging, in three contrasting languages and in machine code | Chapters 19 to 29 |
| **Operating systems** (often missing below university) | Tasks and scheduling; memory protection; file systems; namespaces; devices as files; shared resources and locks | Chapters 7, 9, 10, 12, 14, 17; the `race`, `chorus` and `philo` samples |
| **Networks and the Internet** | Almost nothing: the Hydra has a serial line, not a network. Serial communication, baud and protocols (XMODEM) are the nearest it comes | Sections 5.1, 5.2, 13.6, 14.7 |

| STRAND | WHAT THE HYDRA OFFERS | WHERE |
|---|---|---|
| **Impacts and history of computing** | The 1980s home computer, and why its design still matters; open designs; making versus buying | Chapters 1, 19, 21, 23, 26 (their openings) |
| **Electronics and making** (design and technology) | Soldering; reading parts; power; measuring; working safely and to a standard | Chapters 3 to 5; the Companion's Part One |
| **Music, art and mathematics** | Notes as numbers; music as text; coordinates; turtle geometry; exact fractions; bases | Chapters 15, 16, 18; Labs 12, 13, 15; Projects 2, 3 |

Be honest with yourself about the gap: **networks, the web, data science and security are not here.** The Hydra complements a mainstream course. It doesn't replace one.

## Who it's for

| STUDENTS | WHAT WORKS |
|---|---|
| **Ages 12 to 14** | The emulator; Labs 1 to 9, 12, 13, 15, 20 and 21; HyBASIC as the language; Projects 1, 3 and 4. A group build, with the adults doing the surface-mount chips. |
| **Ages 14 to 18** | Everything. A build in pairs or threes. Two languages, and Part Five for those who want it. |
| **University, first or second year** | A lab companion to a computer organization or operating systems course: Chapters 7, 9, 12, 17 and 26 to 29, with the system's source code as the further reading. Topic 2 has an outline. |
| **Adults and hobbyists** | The manual on its own; the Companion's build log is worth having. |

## What students need first

- **To read fluently**, and to follow written instructions exactly. This matters more than any mathematics.
- **Arithmetic**: the four operations, and the idea of a remainder. Fractions help in Chapter 18. Nothing beyond that until Part Five, which wants powers of two.
- **To type**, even slowly. A student who hunts for every key will struggle with the labs' timings: allow half as long again.
- **No programming.** The manual assumes none.

> **WATCH FOR:** The students who have "done coding" in a block language are often the most thrown by a command line: nothing to click, and it's case-sensitive. Don't skip Lab 3 for them. They need it most.

TOPIC 2

# Course Shapes

Nobody teaches all of it. The manual is a year's reading, and the Companion's labs and projects are about sixty hours at a keyboard, before any soldering. This topic gives you the pieces and their sizes, and then six ways of putting them together, from a single week to a full year.

## The pieces

A **lesson** here means 50 minutes at the keyboard. A lab marked 1 fits one lesson for most students aged 14 and up; allow half as long again for younger students or slow typists. "Reads" is the reading to set *before* the lab.

| LAB | LESSONS | READS | NEEDS | DEPENDS ON |
|---|---|---|---|---|
| 1 Meet the Computer | 1 | Ch 1 | Paper; a Hydra for the last step | |
| 2 A Hydra on Your PC | 1 | 2.1, 2.2 | The web emulator | |
| 3 First Steps | 1 to 2 | Ch 6 | Any Hydra | 2 |
| 4 From Power to Prompt | 1 | 7.1-7.3, 7.5 | Any Hydra | 3 |
| 5 Talking to the Shell | 1 to 2 | 8.1-8.10 | Any Hydra | 3 |
| 6 Scripts | 1 to 2 | 8.11-8.14, 9.7 | Any Hydra | 5 |
| 7 The Tree of Files | 1 | Ch 9 | Any Hydra | 3 |
| 8 Windows | 1 | 10.1-10.4, 10.8 | Any Hydra; a wide terminal | 3 |
| 9 A Command Hunt | 1 | Ch 11 (to dip into) | Any Hydra | 5 |
| 10 Devices Are Files | 1 | Ch 12; 13.4, 13.6, 13.7 | Any Hydra | 7 |
| 11 Disks | 1 | Ch 14 | A Hydra with an SD card | 7 |
| 12 Sound | 1 to 2 | 15.1, 15.2, 15.4, 15.6, 15.7 | Sound: headphones for each student | 3 |

| LAB | LESSONS | READS | NEEDS | DEPENDS ON |
|---|---|---|---|---|
| 13 Pictures | 1 to 2 | 16.1-16.4 | A Vera X (the emulator's will do) | 3 |
| 14 Pins, and the System's Files | 1 to 2 | 17.1, 17.2, 17.5, 17.7, 17.8 | Part A: a real Hydra and an LED | 10 |
| 15 Numbers and Bases | 1 | 18.1-18.4, 18.7 | Any Hydra | 1, 3 |
| 16 HyForth: The Stack | 1 to 2 | 19.1-19.6 | Any Hydra | 3 |
| 17 HyForth: Decisions and Loops | 2 | 19.7-19.17 | Any Hydra | 16 |
| 18 hylang: Expressions | 1 to 2 | 21.1-21.5 | Any Hydra | 3 |
| 19 hylang: Lists and Programs | 2 | 21.6-21.16 | Any Hydra | 18 |
| 20 HyBASIC: First Programs | 1 to 2 | 23.1-23.8 | Any Hydra | 3 |
| 21 HyBASIC: Bigger Programs | 2 | 23.8-23.14 | Any Hydra | 20 |
| 22 Your Language Meets the Hydra | 2 | Ch 20, 22 or 24 | Sound; a Vera X for the last step | 17, 19 or 21; 12 |
| 23 Three Languages | 1 to 2 | Ch 25 | Any Hydra | 17, 19 or 21 |
| 24 The 65C02, on Paper | 1 to 2 | Ch 26 | Paper only | 1, 15 |
| 25 Your First Machine Code | 2 | 27.1-27.9, 27.18 | Any Hydra | 24 |
| 26 Taking Programs Apart | 2 | 28.1-28.3, 29.1-29.7 | Any Hydra | 25 |

| PROJECT | LESSONS | LANGUAGE | NEEDS | AFTER LABS |
|---|---|---|---|---|
| 1 A Quiz | 3 to 4 | HyBASIC | Any Hydra | 20, 21 |
| 2 Turtle Pictures | 2 to 3 | HyForth | A Vera X | 16, 17 |
| 3 A Round for Two Voices | 2 to 3 | MML | Sound | 12 |
| 4 Your Own Command | 2 | rc | Any Hydra | 5, 6 |
| 5 A Program in the Background | 2 | rc | Any Hydra | 4, 6 |
| 6 A Traffic Light | 3 | rc | A real Hydra, LEDs | 14 |
| 7 Faster, with Machine Code | 3 | HyBASIC and assembly | Any Hydra | 21, 24, 25 |

**The build** (the Companion's Part One) is 15 to 20 hours of bench time for one board, however many hands share it, plus the memory card. Topic 6 says how to share it.

## Shape A: a week (emulator only)

A holiday camp, or an activity week: five days, four 75-minute sessions a day. No soldering, no hardware. Ages 12 and up.

| DAY | MORNING | AFTERNOON |
|---|---|---|
| 1. Meet it | Lab 1; Lab 2 | Lab 3 (both sessions) |
| 2. Command it | Lab 5; Lab 9 | Lab 6; Project 4 begun |
| 3. Program it | Lab 20; Lab 21 | Lab 21; Project 1 begun |
| 4. Make it sing and draw | Lab 12; Project 3 | Lab 13; free choice of project |
| 5. Finish | Projects | Lab 4, as a whole-group demonstration; a showcase for visitors |

The week has one language (HyBASIC), three projects to choose from, and something to show at the end of it. Lab 4 comes last on purpose: after a week of *using* tasks and files, the explanation of how it all starts lands on prepared ground.

## Shape B: a nine-week unit, "Inside a Computer"

Two lessons a week, eighteen in all, for a class that has programmed a little somewhere else. Emulator, with a built Hydra on the front bench if you have one. Ages 12 to 15.

| WEEK | LESSON 1 | LESSON 2 | READ BEFORE |
|---|---|---|---|
| 1 | Lab 1 | Lab 2 | Ch 1; 2.1, 2.2 |
| 2 | Lab 3 | Lab 3, and its challenge | Ch 6 |
| 3 | Lab 4 | Lab 7 | 7.1-7.3, 7.5; Ch 9 |
| 4 | Lab 5 | Lab 5; Lab 9 begun | 8.1-8.10 |
| 5 | Lab 9 | Lab 6 | 8.11-8.14 |
| 6 | Lab 10 | Lab 15 | Ch 12; 18.1-18.4 |
| 7 | Lab 20 | Lab 20; Lab 21 begun | 23.1-23.8 |
| 8 | Lab 21 | Project 1 or Project 4 | 23.8-23.14 |
| 9 | Project | Showcase, and the quiz (Topic 16) | |

## Shape C: a semester, "From Chips to Programs"

Eighteen weeks, two or three lessons a week. Emulators for everyone, and one or more real Hydras. Ages 14 to 18.

| WEEKS | WHAT | LABS | PROJECTS |
|---|---|---|---|
| 1 to 2 | Meeting the Hydra | 1, 2, 3 | |
| 3 to 6 | Inside HydraOS | 4, 5, 6, 7, 8, 9 | 4 or 5 |
| 7 to 9 | Devices | 10, 11, 12, 13, 14 | 3 (or 6, with hardware) |
| 10 | Numbers | 15 | |
| 11 to 14 | A language, properly; a second, briefly | Two of 16-17, 18-19, 20-21; then 22, 23 | 1 or 2 |
| 15 to 17 | Machine language | 24, 25, 26 | 7 |
| 18 | Showcase and assessment | | |

Let students choose their main language in week 11, after a taster lesson of each (the manual's section 6.9, and Chapter 25, are made for this). A class with two or three languages in it makes Lab 23 come alive.

## Shape D: a year, with a build

The full course: a Hydra built by the class, and then used for everything. Three terms. Ages 14 to 18, in groups of two or three to a board.

| TERM | BENCH (ONE LONG SESSION A WEEK) | KEYBOARD (ONE OR TWO LESSONS A WEEK) |
|---|---|---|
| 1 | The build: the Companion's Units 1 to 5. Ten to twelve weeks at 90 minutes | On the emulator: Labs 1 to 9, so that students can *use* a Hydra by the day theirs first starts |
| 2 | Hardware projects: Lab 14 Part A; Project 6; an SD card reader for each board | Devices and numbers: Labs 10 to 15; then a language: two of Labs 16 to 21 |
| 3 | Free: a second memory card, a case, a front panel | Labs 22 to 26; a major project of each student's choice; the showcase |

The emulator in term 1 is not a stopgap. It means that the first evening a group's board prints `POST ok`, they already know what to type at it.

## Shape E: a club

One 90-minute meeting a week, with no end date and members of mixed age who come and go. Don't run a course: run a workshop.

- **One build at a time**, on the big table, shared by whoever is there: Topic 6.
- **The lab sheets as badges.** Each member works through them at their own speed, and gets the Companion's Skills Passport signed as they go. Older members sign for younger ones.

- **A project wall.** Everyone always has a project on the go, and shows it when it's done.
- **A demonstration to open each meeting**, five minutes, by a member: one thing I found out.

## Shape F: a university laboratory

Six three-hour laboratory sessions beside a second-year course in computer organization or operating systems. The students can program already. The Hydra's value to them is that the *whole* system is small enough to read: after the labs, the kernel's source is the further reading.

| SESSION | THEME | READ | DO |
|---|---|---|---|
| 1 | A small machine, whole | Ch 1, 7, 26 | Labs 4 and 24; trace the reset sequence in section 7.4 against the boot log |
| 2 | Banked memory and protection | Ch 7; Appendix F | Map a task's address space by experiment: `/proc`'s `mem` and `regs` (section 17.8); explain what stops one task reaching another's RAM |
| 3 | Tasks, scheduling and shared state | Ch 7, 17; Appendix K | The samples `race`, `chorus`, `philo` and `prodcons`; measure the scheduler's time slice; write the race, and its fix, in assembly |
| 4 | Names: everything is a file | Ch 9, 12, 17 | Labs 7, 10 and 14 Part B; build a private namespace; compare with Unix mount tables and with Plan 9 |
| 5 | The machine interface | Ch 27; Appendix K | Lab 25; a filter (like the sample `upper`) written against the system calls directly |
| 6 | Seeing inside | Ch 28, 29 | Lab 26; the bug hunt of section 29.8; then read how `db` is itself only a user of `/proc` (29.11) |

A good assessed exercise to end on: *choose one design decision in HydraOS, find it in the source, and argue for or against it*. Sixteen fixed tasks, bank registers on the zero page, text control files, per-task namespaces: each is a decision with a cost, and the source is short enough to find them in.

## Choosing

| IF YOU HAVE | TAKE |
|---|---|
| A week, and no hardware | Shape A |
| A half-term, and a syllabus to cover | Shape B |
| A semester, and freedom | Shape C |
| A year, a workshop, and a budget | Shape D |
| No timetable at all | Shape E |
| Students who can already program | Shape F, or Shape C at double speed |

Whatever the shape, keep three things: **Lab 3 early** (nothing works without it), **a project for every student** (it's what they'll remember), and **a showcase at the end** (it's why they'll finish).

TOPIC 3

# The Room, the Kit and the Emulator

There are three ways to have a Hydra in the room, and most courses end up using two of them at once.

| | EMULATOR ONLY | ONE BUILT HYDRA, AND EMULATORS | A HYDRA FOR EACH GROUP |
|---|---|---|---|
| What it costs | Nothing but the computers you have | One kit, one set of tools | A kit for every two or three students; tools for each bench |
| Before the first lesson | An hour | The build: 20 hours of yours, or a group's | A term |
| What students miss | The build; real pins, real sound, a real fault | Hands on the hardware, for most of them | Nothing |
| Labs you can't do | 14 Part A; Project 6 | None, in turns | None |
| Best for | Shapes A and B; a first year | Shape C; a first build | Shapes D and E |

The middle column is the one to start with. One real Hydra on the front bench, wired to a projector, changes how a class sees thirty emulators: they stop being a web page, and become *copies of that*.

## The emulator

### Getting it onto every computer

The web emulator is one file, `hydra-16.html`, of about a megabyte. It needs no installation, no network, no account and no plug-in: any browser of the last few years will run it straight from a disk.

1. On one computer with Node.js, make it (the manual's section 2.3): `node reborn/sim/web.js`. This is the only step that needs anything installed, and you need do it only once.
2. Copy `hydra-16.html` to a shared drive, or onto each computer, or onto a memory stick for each student.
3. Open it once on a student computer, with the student's login, *before* the first lesson. School filters and locked-down browsers are the commonest reason for a lost first lesson: find out a week early.

**WATCH FOR:** The emulator keeps each student's SD card inside the browser's own storage, on that one computer, under that one login, for that one copy of the file. A student who sits somewhere else next week, or whose computer "resets itself" overnight (many school computers do), finds an empty card. Decide on day one which of these you'll do about it, and tell the class:

- **Save card** to a file at the end of every lesson, and **Load…** at the start of the next. This is the robust way, and it's a good habit: make it part of packing up.
- Fixed seats, on computers that keep their browser storage. Simplest, when it's true.
- A memory stick each, with the emulator and the card file on it.

### Making the students' Hydras match the book

As it comes, the web emulator's Hydra has a Vera X video card and a clock chip, and the manual's Hydra has neither (the manual's section "This book's Hydra"). The differences are small: a few more start-up lines, today's date, and one more task in `ps`. For Labs 12 and 13 you want the Vera X; for the rest it doesn't matter. Either tell the class to expect it, or have them clear the two boxes in **Setup…** once.

### Giving every student the same starting card

For a lab that needs files of yours on the card (a half-written program, a data file), make a card image once and hand it out: in the terminal emulator's tools, `hydrafs.js` builds one from a folder on your PC (the manual's section 2.6). Students bring it in with **Load…**. The same image, written to a real SD card with `sdwrite.js`, serves a real Hydra.

### The projector

The terminal emulator (`node reborn/sim/run.js -i`) in a large-font terminal window projects better than the web page, and it has two things worth having at the front: `--pc-dir`, so that you can prepare files in your own editor and have them appear on the Hydra as `/pc`; and CTRL-A S, which shows what every task is doing, and is the best single picture of multitasking there is.

## The kit

A kit is planned. Until there is one, this is what one Hydra is made of: price it from the manual's Appendix I, which lists every part with its value and its package, and section 3.9, which says where the parts come from.

| WHAT | HOW MANY | NOTES |
|---|---|---|
| The main circuit board | 1 | Revision 1.8D |
| The parts soldered to it | 216 | 64 chip sockets; 27 resistors and networks; 91 capacitors; headers, connectors, a crystal |
| The four shared RAM chips | (of the 216) | Surface-mount: ask whether they come fitted. If not, an adult fits them (Topic 6) |

| WHAT | HOW MANY | NOTES |
|---|---|---|
| The chips for the sockets | 59 | Five of the ROM sockets stay empty |
| A memory card, with its parts | 1 or more | The through-hole card: 13 parts, 4 RAM chips |
| The four ROM chips, programmed | (of the 59) | Programmed already, or a flash programmer and an hour of your time (section 4.4) |
| An ATX power supply | 1 | Any desktop PC's; second-hand ones are fine |
| A power switch, a reset button, an LED | 1 each | The switch must be one that stays on |
| A USB to RS-232 adapter | 1 | With a 9-pin plug: *not* a "TTL" adapter (section 5.1) |
| An SD card module, and a small SD card | 1 each | A 5-volt module. Not strictly needed, but without it nothing can be saved on the Hydra itself |
| A PC, to be the terminal | 1 | Anything with a USB port |
| For Lab 14 and Project 6 | | LEDs, 2.2K resistors, hook-up wire, a push button |
| A Vera X video card | Optional | Not part of the kit; the emulator has one |

**Spares.** For a class build, hold spares of whatever is cheap and easy to damage: sockets of each size, a few of each resistor and capacitor value, one of each of the common logic chips, and a second set of ROM chips. A whole lesson lost to one bent pin costs more than the spare.

## A bench for soldering

For each pair of students at a bench:

| WHAT | NOTES |
|---|---|
| A temperature-controlled iron, with a stand and a tip cleaner | Not a bare "pencil" iron: the difference in the students' joints is enormous |
| Lead-free solder, 0.5 to 0.8 mm | Topic 4 says why lead-free |
| Safety glasses, for each person | Not optional |
| A heat-proof mat | |
| Flush cutters, needle-nose pliers, tweezers | |
| A magnifier, with a light | One between two is enough |
| Desoldering braid | And a solder sucker for the room |
| Fume extraction | A small bench extractor each, or good cross-ventilation: Topic 4 |

And for the room: two multimeters; an anti-static wrist strap for whoever is fitting chips; isopropyl alcohol and brushes, kept by you; a first-aid kit that someone knows how to use; and running cold water within a few steps.

## The room for keyboard lessons

- **One computer each.** Pairs work for labs if they swap the keyboard at every numbered step; they don't work for projects.
- **Headphones** for Labs 12 and 22, and Project 3. Thirty Hydras playing "Frère Jacques" out of step is a round nobody wants.
- **Paper.** The Companion is meant to be written in, with a pencil. Students who are asked to *predict* on paper before they type learn far more from the same lab than students who just type. If you can't print a Companion each, print the lab sheets as you need them.
- **The manual within reach.** On paper if you can: a student with the manual open beside the keyboard looks things up, and a student who must switch windows to find it doesn't.

TOPIC 4

# Safety

The keyboard lessons need no more care than any computer room. This topic is about the bench: hot irons, molten metal, flux fumes, sharp offcuts, and mains-powered supplies. None of it is exotic, and all of it is managed in school workshops every day. But it has to be managed on purpose.

> **NOTE:** This topic is a starting point for your own risk assessment. It isn't one. Your school, district or institution has rules for hot work, for chemicals and for electrical equipment, and they come first. Take this list to whoever is responsible for workshop safety where you are, before you order anything.

## The hazards, and what to do about each

| HAZARD | HOW IT HURTS | WHAT TO DO |
|---|---|---|
| **The iron's tip**, 350 °C with leaded solder and 370 °C with lead-free | Burns, in an instant | A stand at every station, used every time. Irons carried nowhere: they stay at the bench. Never passed hand to hand. Cables routed away from elbows. Off, and five minutes to cool, before anyone packs up |
| **Molten solder** | Flicks and splashes: burns, and the eyes | Safety glasses on everyone at a bench, and on anyone watching. Never shake or flick an iron. Wet sponges kept damp, not dripping: water on a hot tip spits |
| **Clipped leads** | They fly | Glasses again. Hold the lead while cutting it, or cut into a cupped hand or a box |
| **Flux fumes** | Irritate eyes and lungs; can sensitize, and are a known trigger for asthma | Extraction at each bench, or strong cross-ventilation. Heads to the side of the work, not over it. Ask about asthma before the first session |
| **Lead**, in tin-lead solder | A poison if swallowed: by way of fingers and food | **Use lead-free solder.** With any solder: no food or drink at the bench, and hands washed before leaving the room |
| **Isopropyl alcohol** | Flammable; irritates | You hold the bottle. Small amounts, on a brush, with every iron off and cool. Never near a hot tip |

| HAZARD | HOW IT HURTS | WHAT TO DO |
|---|---|---|
| **The power supply** | Mains voltage inside its case | Cases never opened. Supplies inspected before use: no cracked cases, no damaged leads. The supply unplugged, not just switched off, whenever hands are on the board |
| **A capacitor fitted backwards** | It can burst when the power goes on | The Companion's Unit 2 has every one double-checked. Glasses on for every first power-up, and faces back |
| **Sharp ends** under the board | Scratches | Leads clipped flush. Boards carried by their edges |
| **Static**, to the chips rather than the people | Dead chips, found a week later | A wrist strap for whoever fits chips. Chips stay in their foam until Unit 3 |

## The rules, for the wall

The Companion's Unit 1 has the students tick and sign these. Put them on the wall as well.

1. Glasses on before the iron is on.
2. The iron lives in its stand.
3. If it falls, let it fall.
4. Nothing to eat or drink at the bench.
5. Hair back, sleeves up.
6. Power unplugged before hands touch the board.
7. Wash your hands before you leave.
8. A burn goes under cold running water for ten minutes, and you tell me.

## Before the first session

| ☐ | CHECK |
|---|---|
| ☐ | Your institution's rules for hot work and for soldering by students have been read, and its forms filled in |
| ☐ | A written risk assessment exists, and whoever must sign it has |
| ☐ | Parents or guardians have been told, where that's required, and any asthma or relevant condition is known to you |
| ☐ | You have soldered a board yourself, recently, with the same irons and the same solder |
| ☐ | Every iron has a stand, and every stand is stable |
| ☐ | The extraction works, and is pointed the right way |
| ☐ | There are glasses for everyone, and for a visitor |
| ☐ | You know where the cold tap, the first-aid kit, the fire extinguisher and the isolating switch are |
| ☐ | You know how many students you can supervise at once (below) |

## How many at once

Soldering needs closer supervision than a keyboard lesson. A workable rule: **one adult who can solder for every eight students at benches**, and no student at a bench out of an adult's sight. With a full class and one teacher, that means a rotation: eight at the benches, the rest at keyboards doing a lab, changing over at half time. The year plan in Topic 2 (Shape D) is built on that rotation, and Topic 6 has the details.

Younger students (12 and 13) can solder well, and do. Give them the through-hole resistors, capacitors and sockets; an adult does the surface-mount RAM chips, the first power test, and anything involving the alcohol.

## When something happens

| WHAT | DO |
|---|---|
| A burn | Cool running water, at once, for at least ten minutes. No ice, no creams. Then your institution's first-aid procedure. Anything bigger than a coin, or blistered, is seen by a first-aider or a nurse |
| Solder or a clipping in an eye | Don't rub. Your institution's eye-injury procedure, at once |
| A student feels unwell in the fumes | Out of the room, into fresh air. Stop the session and improve the ventilation before anyone goes on |
| A component smokes or bursts at power-on | Power off at the wall. Everyone back. Let it cool. Then it's a fault-finding lesson: see what it was, and why |
| The smell of burning, and you can't see where | Power off at the wall, and find it before anything goes on again |

Record every incident, however small, the way your institution asks. And talk about it with the class afterwards: a near miss that's been discussed is worth ten rules on the wall.

## The quieter risks

- **Hurry.** Almost every burn, bridge and backwards capacitor happens in the last ten minutes of a session. Stop soldering fifteen minutes before the end, every time, and use the time for inspection and the build log.
- **Pride.** A student who has made a mistake will sometimes hide it. Make finding faults the celebrated thing: the Companion's inspection pages ask "what did the inspection find? (Everybody finds something.)" Mean it.
- **One expert.** In every group, one student soon does all the soldering. Rotate the iron on a timer.



PART TWO

# Teaching It

Notes for every lab, how to run a build with a class, how to assess the work, how to reach the students at both ends of the room, and what to do when the technology lets you down in front of thirty people.


- Topic 5. Lesson by Lesson
- Topic 6. Running a Build
- Topic 7. Assessment
- Topic 8. Reaching Everyone
- Topic 9. When Things Go Wrong in Class

TOPIC 5

# Lesson by Lesson

Notes for each of the Companion's twenty-six labs: what it's really for, how to open it, where students get stuck, and a question to end on. The answers are in Part Three.

## The shape of a lab lesson

The same shape works for nearly all of them, in a 50-minute lesson:

| MINUTES | WHAT | WHY |
|---|---|---|
| 5 | **Open** with a demonstration or a question, at the front (each lab's notes suggest one) | To give the lab a reason |
| 5 | **Predict.** Students read the first table of the lab, and fill in the "guess" column, in pencil, before touching a key | The guess is where the learning is. A wrong guess, corrected by the machine, sticks |
| 25 | **Do.** Students work through the steps. You circulate | |
| 10 | **Questions**, in writing, alone | To find out who understood, and who only typed |
| 5 | **Close**: one student shows one thing; you ask the closing question | To hear it said aloud |

Two habits make the difference between a class that learns from labs and a class that types them:

- **Pencils down, then keys.** Every lab has a prediction to make. Insist on it being written first.
- **"What did it say, exactly?"** When a student says "it doesn't work", don't look at the screen. Ask them to read you the message. Nine times in ten, reading it aloud is the fix.

Reading comes before the lesson, not during it. The Companion names the sections; set them as homework, and open with a two-minute check that they were read.

# Labs 1 to 3: meeting the Hydra

## Lab 1. Meet the Computer

**It's for** the vocabulary (processor, memory, bus, ROM, RAM), and hexadecimal, which everything later leans on.

**Open with** a real board, or the manual's Figure 1-1 on the projector, and the question "which of these chips is the computer?" (None of them is; all of them are.)

**Watch for** hex, which is new and slippery. The commonest error is reading `$10` as ten. Have the class count aloud from `$08` to `$12`. Students who convert by long division get lost; the manual's way, a hex digit for each four bits, is quicker and safer.

**Close with** "why do programmers bother with hex at all?" (Because two hex digits are exactly one byte.)

## Lab 2. A Hydra on Your PC

**It's for** getting every student to a prompt, and the one fact that saves a term's work: where files survive.

**Open with** nothing. Let them open the file and watch the start-up messages scroll. Then ask what they noticed.

**Watch for** the experiment in step 4. Students skip remaking the files after a reset, and then report that the SD card "lost" one. Step 4 is also where the lesson's main point lands, so don't let it be rushed: take the class's results on the board.

**Before the lesson**, read Topic 3's warning about browser storage, and decide how students will keep their cards.

**Close with** "where will you keep your work, from now on?"

## Lab 3. First Steps

**It's for** everything else. A student who can't do this lab can't do any other. Give it two lessons if it needs them.

**Open with** a deliberate mistake at the projector: type `dat`, and ask the class what the Hydra is telling you. Establish at once that an error message is information, not a telling-off.

**Watch for:**

- Capitals. Students arriving from phones type `Ls` and `Date`. The Hydra says `not found`, and they conclude that the Hydra is broken.
- The unfinished definition in step 5. A student who types `: test` and then anything else is inside HyForth's compiler, with a prompt of spaces, and every command they try "does nothing". Teach the cure (`;`) to the whole class, because half of them will meet this by accident within the week.
- `edit`. Some terminals and browsers grab CTRL-O or CTRL-W for themselves. Try the editor on the students' own computers beforehand; in the web emulator, click in the console first.

- Saving into `/ram`. They will. Let one student lose a file to a reset, in public, early: it's the cheapest lesson of the course.

**Close with** "you've typed a command and nothing happens. What three things do you try?" (CTRL-C; CTRL-D; look at the prompt.)

# Labs 4 to 9: inside HydraOS

## Lab 4. From Power to Prompt

**It's for** the idea of a task, and the picture of the system as a few programs co-operating.

**Open with** the terminal emulator at the projector, and CTRL-A S: every task and what it's doing, live. Or with `top` on any Hydra.

**Watch for** the task numbers in step 4. The lab says `kill 2`, but the number is whatever *their* `ps` shows, and killing the wrong one is instructive: kill the shell (task 1), and another starts (the kernel starts `init` again, which becomes it). Let it happen, then ask why.

The reading (Chapter 7) is the hardest in Part Two. Don't set section 7.4 for everyone: the lab and its questions need only 7.1 to 7.3 and 7.5.

**Close with** "is the Hydra really doing sixteen things at once?" (No: one thing at a time, two hundred turns a second.)

## Lab 5. Talking to the Shell

**It's for** variables, redirection and pipes: the grammar of a command line.

**Open with** a question: "how many programs does the Hydra have?" Take guesses, then type `ls /rom/bin | wc -l`, and let the answer raise the next question: what was that `|`?

**Watch for:**

- The two prompts. This lab is typed at rc's `%`, and a student who forgets to type `rc` first finds that `pets=(cat dog fish)` gives an error from HyForth.
- Spaces around `=`. `pets = (cat dog)` isn't an assignment.
- The backquote in `` `{date} ``. It's the key at the top left of most keyboards, and half the class will type an apostrophe.
- **The nine-program limit** on pipes (the manual's section 8.7). Each program in a pipeline is a task, and there are sixteen tasks for everything, so a tenth program gets `no free task`. Nothing breaks: the rest of the line runs without it, and the answer is wrong or missing. It's a good moment to run `ps` and count.

**Close with** "what's the difference between `>` and `|`?" (One sends output to a file; the other to a program.)

### Lab 6. Scripts

**It's for** the step from typing commands to keeping them: the first real programming of the course, for many.

**Open with** "what do you type most often? Wouldn't you rather type one word?"

**Watch for:**

- Tabs and braces in the script. The manual's scripts put the `{` on the `for`'s line and the `}` on a line of its own; rc accepts other layouts, but keep to that one and the shape stays easy to see. The editor's auto-indent helps.
- Step 5's question has a sting: the script copied to `/bin` is really in `/ram/bin`, and is gone at the next reset. Let them find that out next lesson, and then teach `/sd/0/bin`. (A directory made there joins `/bin` the next time the namespace is built: after a reset, or `newns`.)

**Close with** "when is a script better than a function?" (When you want it tomorrow.)

### Lab 7. The Tree of Files

**It's for** paths, and the Hydra's best idea: one tree of names for everything.

**Open with** a drawing on the board of the tree as far as the class can dictate it from Lab 3, and add to it as the lesson goes.

**Watch for** relative paths. `cd ../songs` is the first time many students meet `..`, and the commonest wrong picture is that `..` means "back to where I was before". It means "the directory above".

Step 6 (`bind`) is the conceptual high point. A student who sees that `/mnt` showed the songs *without anything being copied* has understood what a name is.

**Close with** "is `/dev/time` a file?" (It behaves like one. That's all "file" means, here.)

### Lab 8. Windows

**It's for** seeing multitasking, rather than being told about it.

**Open with** two windows side by side at the projector, a slow command running in one while you type in the other.

**Watch for** the window key. CTRL-] is awkward on some keyboard layouts (on many European layouts `]` needs another modifier), and a few terminal programs take it for themselves. Try it on the students' machines first. (Section 10.3 lists the window key's commands; CTRL-] twice sends one through.)

Students with small browser windows can't see tiled windows properly. Set the console's size to at least 80 columns first.

**Close with** "you have two windows. How many shells are running? How many tasks?"

### Lab 9. A Command Hunt

**It's for** learning to look things up: the skill that outlasts every particular command.

**Open with** nothing but the rule: the manual is allowed, asking the teacher is not.

**Watch for** students who guess options from other systems (`head -n 3`, `grep -r`). The Hydra's commands are smaller than Linux's, and the usage line is the authority: type the command wrongly, and it prints its usage.

This lab works well as a race in pairs, one with the manual and one at the keyboard, swapping at half time.

**Close with** "which command did you find that you didn't know you wanted?"

## Labs 10 to 14: devices

### Lab 10. Devices Are Files

**It's for** the idea that reading and writing files is the *only* interface there is.

**Open with** `echo b115200 >/dev/serctl` on a real Hydra at the projector, and the terminal turning to nonsense. (Then change the terminal's speed, and it comes back.) A device was reconfigured by writing a word into a file.

**Watch for** step 5, the escape sequences: this is the first HyForth that students type, and they must type it *exactly*. The space after `."` is the usual casualty. Tell them it's a spell for now; Lab 16 explains it.

**Close with** "what would you have to know to control a device nobody's told you about?" (The names of its files, and the words its control file understands.)

### Lab 11. Disks

**It's for** what "saving" really means: where bytes go, how much room they take, and what survives.

**Open with** the question "your file is two bytes long. How much room does it take up?" and let step 3 answer.

**Watch for** the emulator: its SD card "is" a file on the PC, and students ask whether it's real. It's a good question. The honest answer is that the Hydra can't tell, and that this is what "emulate" means.

**Close with** "which of the Hydra's disks would you trust with your project? In what order?"

### Lab 12. Sound

**It's for** fun, mostly, and it earns its place: it's the lab that students show their friends. Underneath, it's about representing something that isn't a number (a tune) as numbers and text.

**Open with** `play /rom/songs/scom.zsm`, loud.

**Watch for:**

- No sound. In the web emulator the **Sound** button must be pressed (browsers won't play until the user clicks), and the speed must be **Real time**. On a real Hydra, check the plug is fully in J26.
- Headphones. Insist.

- Bars that don't add up, in step 6. A student whose two voices drift apart has miscounted beats: have them count each bar aloud.

Students who play an instrument will race ahead; pair them with those who don't, and let the musician dictate while the other types.

**Close with** "what's the difference between writing down a tune and recording one?" (The score, and the ZSM file: a description, and a list of events.)

### Lab 13. Pictures

**It's for** coordinates, and again the same idea: a picture described in text.

**Open with** the question "where is the point (0, 0)?" Most will say the bottom left, from mathematics. On a screen it's the top left, and *y* grows downwards.

**Watch for** the house in step 5. Students who plan on squared paper first finish in half the time of those who type and hope: hand out squared paper.

When the bitmap goes on, the console's text becomes 40 columns wide. It isn't broken, and `bitmap off` puts it back.

**Close with** "why keep the drawing as a file of commands, when you could save the picture?"

### Lab 14. Pins, and the System's Own Files

**It's for** the moment when software moves the physical world: Part A. And for seeing the operating system through its own files: Part B.

**Open with** an LED on the front-bench Hydra, lit by `echo 0 >/dev/gpio/2` at the projector.

**Watch for:**

- The wiring. Power *off* while wiring. An LED straight across a pin and ground, with no resistor, or a wire that shorts +5 V to ground: check each circuit before its power goes on. The manual's caution in section 17.1 is not decoration.
- "It lights when I write 0." Yes: the LED is wired from +5 V to the pin, so the pin has to go *low* to light it. The manual explains why (a pin can sink more current than it can supply). It's worth five minutes at the board.
- On an emulator, Part A stops at step 2. Those students go straight to Part B.

**Close with** "the LED, the clock and the task list are all files. What isn't a file, on the Hydra?"

## Labs 15 to 23: programming languages

### Choosing the languages

Few courses have time for all three. Some advice:

- **One language, for everybody: HyBASIC.** The gentlest start, readable programs, and exact fractions that delight mathematics teachers. Labs 20 and 21.
- **A second, for contrast: HyForth.** It's the shell, so it costs no extra set-up, and nothing else shows so plainly that a language is a *way of thinking*. Labs 16 and 17.
- **hylang** for a class that already programs, or one with a mathematical bent. Its ideas (functions as values, lists, recursion) are the ones a later computer science course will build on. Labs 18 and 19.

If you let students choose, do it after a taster of each: the manual's section 6.9 is ten minutes, and Chapter 25 sets the three side by side.

### Lab 15. Numbers and Bases

**It's for** breaking the belief that a computer's numbers are the calculator's numbers.

**Open with** "what's 0.1 + 0.2?" Have someone try it in a spreadsheet, or in another language, if one's to hand, and then on the Hydra.

**Watch for** the quotes around `calc`'s sums. Without them, the shell takes `*` and `(` for its own, and the error comes from rc, not from `calc`.

**Close with** "is a third a number the Hydra can hold exactly? Is the square root of two?"

### Lab 16. HyForth: The Stack

**It's for** the stack: a simple idea that takes an hour to believe.

**Open with** a real stack: five numbered cards or plates. Act out `1 2 3 swap` with a volunteer. Then act out `+`, with the volunteer taking two cards and putting one back. It looks childish, and it works on university students too.

**Watch for:**

- Missing spaces: `2 3+`. The message `3+ ?` is HyForth's "I don't know that word".
- Leftovers. A student whose answers are wrong "for no reason" has numbers on the stack from an earlier line. Teach `.s` as the first thing to type when puzzled.
- Step 1's table is the lab. Don't let them type before they've filled in the guesses.

**Close with** "why doesn't Forth need brackets?"

### Lab 17. HyForth: Decisions, Loops and Programs

**It's for** building a program out of small tested words: the most transferable habit in programming.

**Open with** FizzBuzz, played aloud round the room, to 30. Then: "how would you teach the Hydra to play?"

**Watch for:**

- `if` at the prompt (`if: compile only`). Decisions and loops live inside definitions.

- Stack imbalance across `if` … `else` … `then`: one branch uses the number and the other doesn't. The discipline that cures it is the stack diagram: have them write one for every word, and check both branches against it.
- `do` loops with the limit and the start the wrong way round.

**Close with** "you've written `triangle` using `stars`. Which did you test first? Why?"

### Lab 18. hylang: Expressions and Functions

**It's for** the idea of a *value*: everything you type is worth something.

**Open with** `(+ 1 2 3 4 5)`, and "what's odd about that plus sign?"

**Watch for:**

- Parentheses, of course. Teach counting them from the inside out, and teach the prompt of spaces as the sign of one missing.
- Braces against parentheses in `fun`. The body goes in braces because it mustn't be worked out yet. A body in parentheses gives `Unbound Symbol 'n'` at once, which is baffling until explained.
- `(square)` returning a function. Students think it's an error. It's one of the best ideas in the language, and a quick student can be set loose on it.

**Close with** "what's the value of `(def {x} 5)`? And of `x`, afterwards?"

### Lab 19. hylang: Lists, Decisions and Programs

**It's for** working on a whole collection at once, and for recursion.

**Open with** "add up the numbers from 1 to 100", by any means, timed. Then `(sum (range 1 101))`.

**Watch for** recursion without a stopping case. `fact` without its `if` runs until memory is gone. Ask every student, of every recursive function: "which line stops it?"

**Close with** "you wrote no loop in step 3. Where did the loop go?"

### Lab 20. HyBASIC: First Programs

**It's for** the fundamentals: a variable, a sequence, a loop, a decision, input and output.

**Open with** a three-line program that asks your name and insults you politely. Then change one line, in front of them, and run it again. The edit-run cycle is the lesson.

**Watch for:**

- `AGE = AGE + 1` read as an equation ("that's impossible"). Say "becomes", never "equals".
- A variable's name misspelled. It isn't an error in BASIC: the new name is a new variable, worth 0. The symptom is a mysterious 0.
- Students who learned to renumber by retyping. Show them that a line typed again replaces the old one, and that `EDIT` exists.

**Close with** "what's the difference between `PRINT A` and `PRINT "A"`?"

### Lab 21. HyBASIC: Bigger Programs

**It's for** structure: loops that end on a condition, data in arrays, and procedures.

**Open with** the question "how many times do you have to double 1 to pass a million?" Take guesses. (Twenty.)

**Watch for** whose variable it is. A `SUB`'s or a `FUNCTION`'s variables are its own (the manual's section 23.12), so a student who adds to `TOTAL` inside a `SUB` finds the main program's `TOTAL` unchanged, and decides that HyBASIC is broken. `SHARED TOTAL` in the `SUB` is the cure; passing the value in and giving the answer back with a `FUNCTION` is the better one. The lab's programs have no line numbers: they're kept in files and run from the shell with `basic file.bas`. Make that the class's habit from this lab on.

**Close with** "what does a `SUB` give you that a `GOSUB` doesn't?" (Its own variables, and a name.)

### Lab 22. Your Language Meets the Hydra

**It's for** using a manual as a programmer does: the job is set, and the student finds the means.

**Open with** the list of seven jobs, and nothing else. This lab is deliberately unscaffolded.

**Watch for** frustration, at first, from students used to being given the code. Hold your nerve. Point at the chapter, not at the line.

Mixed-language classes can compare their answers to each job: it becomes Lab 23 by itself.

**Close with** "which job did your language make easy? Which did it make you work for?"

### Lab 23. Three Languages

**It's for** reading code you didn't write, in a language you don't know: what programmers actually do all day.

**Open with** the guessing game, played against the class at the projector. Then ask for the best strategy, and why seven guesses is always enough for a hundred numbers.

**Watch for** step 6, where HyForth counts to 40,000 and prints -25536. It's the 16-bit cell overflowing into the sign bit, and a perfect, unplanned-looking introduction to Lab 24.

**Close with** "is there a best programming language?"

## Labs 24 to 26: machine language

### Lab 24. The 65C02, on Paper

**It's for** the bottom layer: there is nothing underneath this but electronics.

**Open with** "be the processor": three students at the front, as A, X and the program counter, each holding a card with a number, and a fourth reading out instructions.

**Watch for:**

- `#.` `lda #$03` loads the number 3; `lda $03` loads whatever is in memory at address 3. It's the classic beginner's bug, and Lab 25 has them make it on purpose.
- The carry. `$FF + $01` gives `$00` and a carry of 1: an odometer rolling over.
- `bne` after `dex`: "branch if the result wasn't zero". Students need to say that sentence aloud a few times.

No Hydra is needed, which makes this a good lab for a day when the computers are down.

**Close with** "the 65C02 has no multiply instruction. Is that a problem?"

### Lab 25. Your First Machine Code

**It's for** the whole cycle: source, assembler, program, run.

**Open with** `./stars`, and then `xd stars`: the program as bytes. "Somewhere in there is the instruction that prints a star. Next lesson we'll find it."

**Watch for** typing. `stars.s` is thirty lines, and one wrong character stops it. Have students assemble after the first few lines to catch errors early, or give the file out on a card image (Topic 3) to those who would otherwise lose the whole lesson to typing.

Step 4, the deliberate mistakes, is the centre of the lab. The one that assembles without complaint (`lda 1`) matters most: the assembler can't tell you about a program that's valid but wrong.

**Close with** "what's in `stars.s` that isn't in `stars`?" (The names and the comments: everything that's for people.)

### Lab 26. Taking Programs Apart

**It's for** closing the loop: the program from Lab 25, opened up and watched from inside.

**Open with** the single-step at the projector: `db ./stars`, and `s`, `s`, `s`, with the class calling out what A will hold next. They traced by hand in Lab 24; now the machine shows its working.

**Watch for** `c` with no breakpoint set: the program runs to its end, `db` exits, and the student thinks it crashed.

In step 8, the students change a running program's memory. Make sure the questions that follow are answered: the file on disk hasn't changed, and neither has the source.

**Close with** "you've now seen the same program as BASIC-like source, as assembly, as bytes, and running. Which one *is* the program?"

TOPIC 6

# Running a Build

A Hydra is 2,234 solder joints on the main board, 240 on a memory card, 59 chips to seat, and a day or two of fault-finding. One determined adult can do it in a week of evenings. A class can do it in a term. This topic is about the term.

## Is the class ready?

Don't begin on the Hydra. Begin on something that doesn't matter.

| BEFORE THE HYDRA | WHY |
|---|---|
| Every student has made twenty joints on scrap, to the standard of the manual's Figure 3-3 | The Hydra's board is not for learning on |
| Every student has cleared a bridge, and removed a part, on scrap | They will need to, and the first time shouldn't be on a 62-pin slot |
| Every student has signed the Companion's safety rules (Unit 1), and so has an adult | Topic 4 |
| You have built a Hydra yourself, or at least soldered a hundred of its joints | You can't supervise what you haven't done |

An inexpensive "learn to solder" kit each, built in the week before, does all of the first three rows, and gives every student something to take home.

## Sharing one board

A board has room for one iron at a time. So a build shared by a class is a relay, and the question is how to make the relay fair, and how to make the board come out working.

**The team.** Two or three students to a board is ideal. With one board for a class, run it as shifts of three: one soldering, one placing parts and holding the board, one reading the manual aloud and ticking the Companion. Rotate the three jobs every fifteen minutes, on a timer.

**The reader matters as much as the solderer.** The commonest cause of a dead Hydra isn't a bad joint: it's a wrong part in the right place. The student with the manual, checking each value against the table before it goes in, is the quality control. Say so, often.

**One log for the board.** If several groups share one build, the Companion's Units 2 to 5 are kept as *one* copy, which lives with the board: who fitted what, on which day, checked by whom. Each student still keeps their own logbook.

## The order of work, with a class

The manual's order (lowest parts first) is right. Here it is as sessions of 90 minutes, with what to watch at each. The bench time is about 20 hours for a first build; with a relay, allow twelve sessions.

| SESSION | WHAT GETS DONE (THE COMPANION'S UNIT 2) | JOINTS | WATCH FOR |
|---|---|---|---|
| 0 | Practice kits; safety; the kit counted (Unit 1) | | The kit's count: find missing parts *now* |
| 1 | Step 1, by an adult: the four surface-mount RAM chips. Step 2: resistors | 210 | Resistor values: have every one read aloud, and measured if in doubt |
| 2 | Step 3: the 78 ceramic capacitors | 156 | Tedium. This is the session where joints get careless. Short shifts |
| 3 to 6 | Step 4: the 64 sockets | 1,282 | Orientation, every single socket. A socket in backwards still works, but the chip then goes in backwards. Unsoldered pins: a 40-pin socket with 39 joints looks finished |
| 7 | Step 5: the crystal, the reset chip, headers and pin sockets | 148 | J1 and J2 dead straight. U43 the right way round |
| 8 | Step 6: the electrolytic capacitors. Step 7 begun: J3, J26, J11 | 66 | **Polarity.** Stop the room, and check all eleven polarized capacitors together before any is soldered |
| 9 | Step 7: the six slots | 372 | Slots sitting flat. Bridges between the slots' close pins |
| 10 | Step 8: inspect and clean. The first power test (Unit 3) | | Everyone inspects. You do the power test, with glasses on and faces back |
| 11 | The memory card (Unit 4). Chips in (Unit 3) | 240 | The card's J1 and J2 upside down, soldered in place on the main board. Chip families: 74F is not 74LS |
| 12 | First switch-on (Unit 5) | | See below |

## The surface-mount chips

The four shared RAM chips, U25 and U27 to U29, have 32 legs each at a spacing that needs flux, a clean fine tip, magnification, and a steady hand. They're also Step 1, when the class has had the least practice.

Do them yourself, before the first session, or have them done. It isn't cheating. It's the difference between a board that starts and a board that teaches despair. A confident older student can do the fourth one, watched, once the first three are down.

## The first switch-on

Make an occasion of it. Everyone who worked on the board should be in the room.

1. **Before the power:** the Companion's "Before the switch" list, read aloud, each item answered by a named student who goes and looks.
2. **The terminal first,** on the projector, at 9600 baud. Then the power.
3. **If it starts:** let the students read out the messages. Type `free`. Then hand the keyboard to the class.
4. **If it doesn't,** and it very often doesn't: that's the next two lessons, and they're good ones. Say out loud, right away, that this is normal and expected, and that now the interesting part starts.

## Fault-finding as a lesson

A board that doesn't start is the best teaching material you'll have all year. Treat it that way.

- **One fault diary, on the board** (the Companion's Unit 5 has one; copy it large). Every check that anyone makes is written up, with what was found. Nothing is tried twice. Nothing is tried without being written down first.
- **One change at a time.** The student who reseats six chips "to be safe" has destroyed the evidence. Make the rule explicit.
- **Divide the problem.** The manual's section 5.7 is in the right order: power, then the serial line, then the chips the Hydra needs to start, then the address decoder, then joints. Have the class say what each check *rules out*.
- **The usual suspects**, roughly in the order they turn out to be the answer: a chip in the wrong socket or the wrong way round; a bent leg folded under a chip; a socket pin never soldered; a solder bridge on a slot or a socket; the wrong serial adapter (TTL instead of RS-232) or the wrong baud rate; a clock jumper missing or doubled; a ROM chip not programmed, or in the wrong socket.
- **When it starts but `POST` reports faults,** the numbers on the `POST` and `RAM` lines point at a chip: the manual's section 5.7 has the table, and section 7.4 explains every field. This is where the students who read Chapter 7 properly get to be the experts.
- **Then the hardware test** (`hwtest`, or T held at reset), which tests each part in turn and names what fails.

Budget two sessions for faults. If the board works first time, spend them on the SD card reader and Lab 14.

## What the students keep

A single class Hydra belongs to the room. That's a pity, and worth softening:

- A photograph of each student with the board, at the first prompt.
- Their name on a label under the board, or in a file on its SD card that says who built it.
- The Companion's signed "this Hydra started on" line.
- Their own SD card, or card image, with their work: the same files run on the emulator at home.

## A build without a class

For a club (Topic 2's Shape E) or a single group, drop the sessions and keep the rules: practise first; one person reads while another solders; tick the Companion as you go; stop fifteen minutes before you have to; inspect with someone else's eyes; and write the fault diary even when you're sure you'll remember.

TOPIC 7

# Assessment

Three kinds of work come out of a Hydra course: lab sheets, projects, and (perhaps) a build. Each wants a different kind of marking. This topic has a way to do each, with rubrics you can copy; Topic 16 has a bank of quiz questions with answers.

## What to assess, and what not to

**Assess understanding, not typing.** A lab's "Do" steps can be completed by a student who understands nothing: the table fills in itself. The marks are in the predictions, the written questions and the "check yourself" list.

**Assess the process as well as the product.** A project that works, with an empty logbook, has taught less than one that half works, with three pages on why. The fault diary of a failed build is worth more than a Hydra that happened to start.

**Don't assess speed.** The fast typists aren't the deep thinkers, and the labs' times are guides.

## Lab sheets

Marking twenty-six labs for thirty students in full is not possible, and not needed. Do this instead:

- **Every lab: a glance**, in the lesson, while you circulate. Are the predictions filled in, in pencil, before the answers? Tick the sheet. This takes seconds, and it's what keeps the habit alive.
- **Every lab: self-checking.** Students tick their own "check yourself" lists. Collect the unticked boxes, on a show of hands: they tell you what to reteach.
- **A sample, marked properly**: one question from each lab, chosen by you after the lesson, for every student; or the whole of one lab in four, in rotation. Part Three has the answers.

A four-level scale is enough for a written answer:

| LEVEL | THE ANSWER |
|---|---|
| 3 | Is right, and says *why*, in the student's own words |
| 2 | Is right, but only states the fact, or copies the manual |

| LEVEL | THE ANSWER |
|---|---|
| 1 | Has the right idea, with a mistake; or is right about what happened, and wrong about the reason |
| 0 | Is missing, or shows no understanding |

The difference between 2 and 3 is the one that matters. "`/ram` loses files at a reset" is a 2. "`/ram` is made of memory, and memory is wiped when the Hydra starts, so anything only there is gone" is a 3.

## The Skills Passport

The Companion's last pages list 44 skills, each with a place for a signature. Use it as a practical assessment that runs all course long:

- A skill is signed when the student **does it in front of someone**, without notes: you, or a student who already holds that signature.
- Peer signing works, and it's how you find time. Spot-check one in five.
- The "Understanding" skills are signed after a spoken explanation of a minute or so. These are the hardest to earn, and the best evidence you'll get.

## Projects

A project is assessed on four things. Share the rubric with the students before they start.

| | 4 | 3 | 2 | 1 |
|---|---|---|---|---|
| **It works** | Does everything it set out to; copes with bad input | Does what it set out to, with ordinary input | Runs, with faults | Doesn't run, or is the starting listing unchanged |
| **It's the student's own** | Four or more changes from the starting point, one of them the student's own idea | The changes asked for | One or two small changes | None |
| **It can be read** | Clear names, comments that say *why*, tidy layout; built from small pieces, each tested | Comments and layout good enough for a classmate to follow | Hard to follow | No comments; one lump |
| **It can be explained** | The student explains any line asked about, and what they'd do differently | Explains how it works overall | Explains what it does, not how | Can't explain it |

"It can be explained" is assessed by talking: three minutes at the student's own screen. Ask two questions: *"what does this line do?"*, pointing at one that isn't trivial; and *"what was the hardest bug, and how did you find it?"* The second question, asked every time, teaches students to notice their own debugging.

**The showcase.** End every course with one. Each student has three minutes to show one thing running to people who haven't seen it: another class, parents, the head. It isn't marked. It's what makes them finish.

## The build

A build is group work on a single object, so assess the individual's part in it through the records:

| | EVIDENCE | LOOKING FOR |
|---|---|---|
| **Safe working** | Your own observation; the signed rules | Glasses on, iron in its stand, without being told. This is pass or fail, and failing means off the bench until it's put right |
| **Workmanship** | The student's own practice board; their signed rows in the Companion's Unit 2 | Joints to the manual's standard; parts the right way round; faults found and repaired |
| **Method** | The fault diary; the student's logbook | One change at a time; results written down; a reason given for each check |
| **Understanding** | A spoken question at the bench | "What is this part for?" "What would happen if it were in backwards?" "What did that test rule out?" |

Never mark a student down because the group's board didn't start. Mark what they did about it.

## A written test

If you must have one, Topic 16 has sixty questions, in six sets of ten, with answers. They're short-answer questions, not multiple choice: a student who can write "the kernel gives each task a turn, two hundred times a second" has shown something that choosing (c) doesn't.

Better than a test, if your timetable allows: **a practical.** Twenty minutes at a Hydra, with the manual allowed, and five tasks on a card. For example, after Shape B:

1. Make a directory `/sd/0/test`, and in it a file with your name in it.
2. How many lines in `/sd/0/sample/as/tick.s` contain the word `SLEEP`? Write the command and the answer.
3. Start `sleep 300` in the background; find its task number; stop it.
4. Write a three-line script that prints the date, then how many programs are in `/rom/bin`.
5. Write a HyBASIC program that prints the 9 times table.

Using the manual is the point, not a concession: looking things up *is* the skill.

## Reporting

For a report home, three sentences cover it: what the student can now *do* (from the Passport), what they *made* (the project, by name), and how they *work* (from the logbook and the fault diary). Parents understand "built a working computer, wrote a quiz game in BASIC, and found a fault by methodical testing" better than any grade.

TOPIC 8

# Reaching Everyone

Every class has a student who finished Lab 3 in ten minutes and one who is still looking for the CTRL key. The Hydra is unusually good at holding both, because there is always a level below to go down to, and always a chapter that can wait. This topic is about using that.

## The student who races ahead

Don't give them more of the same. Give them something deeper.

- **Every lab's challenge** (the ★ box) is written to be genuinely harder, not just longer. Some of them (primes in HyForth, the Hydra guessing *your* number) will hold a strong student for a lesson.
- **Send them down a layer.** Finished the HyBASIC labs? Read Chapter 26, and do Lab 24. The quick student in a HyBASIC class who turns up with a working `ASM` block has learned more than six extra BASIC exercises would teach.
- **Send them sideways.** A second language (Lab 23's challenge: do it again in another) is real stretch, and it can't be done by typing faster.
- **Make them the expert.** One student who knows `db`, or the window keys, or MML, and who the others must ask. Explaining is the best test of understanding there is, and it frees you.
- **The source.** For the rare student who wants it: HydraOS's whole source is in the repository, with comments, and the manual's Appendix K lists the system calls it implements. "Find where `ps` gets its list from" is a week's happy digging.

> **WATCH FOR:** The fast student who is fast because they skip the predictions and the questions. Speed through the "Do" steps proves nothing. Look at their written answers before you hand out an extension.

## The student who is struggling

Find out which kind of struggle it is. They need different help.

| THE TROUBLE | WHAT IT LOOKS LIKE | WHAT HELPS |
|---|---|---|
| **Typing** | Everything takes three times as long; many errors that are slips, not misunderstandings | Pair with a typist, swapping roles. Teach ↑ to recall and repair a line. Allow the editor and paste for long listings: a card image with the starting files (Topic 3) |
| **Reading** | Can do it when shown, not from the sheet | Read the step aloud together. The labs' steps are short on purpose. For the manual's reading, set less: each lab's notes in Topic 5 say which sections are essential |
| **Precision** | "It doesn't work" twenty times a lesson: capitals, spaces, the wrong quote | The read-it-aloud habit: "what did you type, exactly? What did it say, exactly?" Card 1 beside the keyboard |
| **The idea itself** | Types correctly, can't predict | Go back to the physical model: cards for the stack (Lab 16), a drawn tree for files (Lab 7), students acting the registers (Lab 24) |
| **Confidence** | Won't press RETURN without checking with you | Say, and show, that nothing typed can break it: break things at the projector, on purpose, and reset. The manual's section 6.1 says the same |

For a student well behind, cut the course rather than rush it. Labs 1, 2, 3, 5, 7, 20 and 21, and Project 1, are a complete and honourable course on their own.

## Younger students (12 and 13)

- **HyBASIC**, and only HyBASIC. It reads like English, and its error messages are words.
- **Shorter sessions at the keyboard**: a lab split over two lessons, with the questions as the second.
- **The emulator**, with the Vera X on: sound and pictures (Labs 12 and 13) are the hooks at this age.
- **In a build**: through-hole parts only, in threes, with an adult for every six, and the adult doing Step 1, the alcohol, and the first power-up. They solder as well as sixteen-year-olds after the first evening. They need more help *reading* the parts.
- **Leave out** Chapter 7's detail, namespaces (the second half of Chapter 9), and Part Five, unless a student asks for them. Some will.

## Older students and adults

- **Skip nothing in Chapters 7 and 9.** These are the chapters that an older student finds genuinely new: most have programmed, and few have ever seen how an operating system starts or what a namespace is.
- **Two languages, at least**, and Lab 23.
- **Part Five for everyone.**
- **The design questions.** Why sixteen tasks? Why are devices files? What would you change? Topic 2's Shape F has more.

## Students who can't solder

Some students can't, or shouldn't: a tremor, a visual impairment, a respiratory condition, or simply a refusal that you've decided to respect. A build has plenty of real work away from the iron:

- **Reader and checker**: reading each step and each part's value aloud, and ticking the Companion. It's the job that decides whether the board works (Topic 6).
- **Inspector**: the magnifier, and the final say on whether a joint passes.
- **Kit manager**: sorting, counting, identifying parts, and handing over the right one.
- **Tester**: the multimeter in the power test, and the terminal at switch-on.
- **Fault diarist**: the keeper of what's been tried.

Rotate everyone through these jobs, so that they're roles in a team and not a consolation.

## Seeing, hearing and moving

- **Colour.** Resistor values are in coloured bands, and about one boy in twelve can't tell some of them apart. The manual's tip is the answer for everyone: measure it with the multimeter. Never make reading the bands by eye the only way.
- **The terminal's colours** can be changed in most terminal programs; the Hydra's own text needs no colour to be understood. Lab 10's colour questions can be answered from the escape sequence alone.
- **Small text.** The web emulator's console scales with the browser's zoom. On a real Hydra, the terminal program's font is yours to choose.
- **Sound.** Labs 12 and 22 and Project 3 have a visible side: the score is text, and a tune that's wrong can be seen to have bars of the wrong length. A deaf student can do all of the notation, and check it by counting.
- **Fine movement.** Soldering is covered above. At the keyboard, sticky keys (the operating system's own accessibility setting) make CTRL-] and the other two-key combinations possible with one hand.

## Language

The manual is written in short sentences and plain words on purpose, and it defines every technical term where it first appears (in bold) and again in Appendix J. For students working in a second language:

- **The glossary first.** Appendix J is a word list as much as a glossary. A page of it for homework, before each part, helps more than anything.
- **The Hydra's own words are few.** About fifty commands, each a short English word or an abbreviation. A card of the twenty on Card 2, with translations written beside them by the student, is enough for months.
- **Error messages are the hard part**, because they're terse. Lab 3's table of mistakes, and the "When it goes wrong" tables on Cards 4 to 6, turn them into a list that can be learned.

## Girls, and everyone else who's been told this isn't for them

A few habits make a room where everyone stays, and none of them cost anything:

- **Pairs, by your choice, not theirs**, and changed often.
- **Rotate the iron and the keyboard on a timer**, so that nobody is ever "the one who watches".
- **Projects with a choice of subject.** A quiz about anything; a tune of their own; a picture of their own. The starting listings were chosen so that the subject is open.
- **Praise the method, not the talent.** "You found that by changing one thing at a time" tells a student what to do again. "You're a natural" tells the others that they aren't.

TOPIC 9

# When Things Go Wrong in Class

Things that will happen with thirty students and thirty Hydras, and what to do about each, quickly. Hardware faults in a board you've built are the manual's section 5.7 and Topic 6; this topic is about the lesson.

## The emulator

| WHAT HAPPENS | WHY | WHAT TO DO |
|---|---|---|
| The page opens, but nothing appears in the console | The browser blocked the page's scripts, or it's a very old browser | Try another browser. Some school set-ups block a page opened from a network drive: copy it to the computer's own disk, and try again |
| A student's files have gone since last lesson | Another computer, another login, or the browser's storage was cleared (Topic 3) | Nothing brings them back. **Save card** at the end of every lesson, from now on |
| Typing does nothing | The console doesn't have the keyboard | Click in the console |
| CTRL-W closed the page; CTRL-N opened a window | The browser took the key before the page saw it | There's no cure inside a browser tab. Find out beforehand which keys your browser takes, and warn the class; or use the terminal emulator, which doesn't have the problem |
| No sound | The **Sound** button hasn't been pressed; or the speed isn't **Real time**; or the computer is muted | In that order |
| Music plays too fast | The speed is 2×, 4× or **As fast as it goes** | **Real time** |
| It's very slow | An old computer running at **Real time** can't keep up: the page shows how close to real time it's running | Close other tabs. Turn off the Vera X in Setup if the lab doesn't need it |

| WHAT HAPPENS | WHY | WHAT TO DO |
|---|---|---|
| The start-up messages don't match the book | The Vera X and the clock chip (the manual's "This book's Hydra") | Nothing. Tell them it's expected |
| A student pressed **New** on their SD card | It's empty now | **Samples** brings the sample card back (and loses what was there). **Load...** brings back a saved card |

## At the prompt

| WHAT A STUDENT SAYS | WHAT'S HAPPENED | WHAT TO DO |
|---|---|---|
| "Nothing I type does anything" | A program is running and waiting; or they're inside a HyForth definition (a prompt of spaces); or inside a quote | CTRL-C; then CTRL-D; then `;`; then reset. In that order |
| "It says not found" | A misspelling; a capital; or the file is in another directory | "Read me what you typed." Then `ls`, to see what *is* there |
| "My prompt looks different" | They're in rc (`%`), HyBASIC (`>`), hylang, `ed` (`*`) or `db` | Card 1's table: each prompt, and how to leave |
| "It printed a question mark" | HyForth doesn't know a word: usually a missing space | Look for two things run together: `3+`, `."Hello` |
| "My program has gone" | It was in `/ram`, and the Hydra was reset; or it was only in HyBASIC's memory, never saved | Sympathy, and then the rule: `/sd/0/work` |
| "`edit` won't let me out" | CTRL-C doesn't leave the editor, on purpose | CTRL-X, then Y or N |
| "The screen is a mess" | A program printed binary, or two tasks wrote at once | CTRL-L redraws the editor. Otherwise open a new window (CTRL-] C), or reset: a reset cures everything |
| "It says `no free task`" | All sixteen tasks are in use: a pipeline of ten programs (the manual's section 8.7), or programs left running in other windows | `ps` shows them. Use fewer programs in the line, or end what's still running (`kill`) |
| "It says `duplicate definition`" | A HyBASIC program with two `SUB`s or `FUNCTION`s of one name (section 23.13) | `LIST`, find the two, and rename one |
| "I pasted my program and bits of it are missing" | The terminal program's flow control isn't XON/XOFF, so it didn't wait when the Hydra asked (the manual's sections 5.2 and 13.6) | Set it to XON/XOFF. Or paste a screenful at a time; or bring the file over with `/pc` |
| "The date is wrong" | No clock chip: it's the year 2000 | `echo` the date into `/dev/time` (Lab 14), or ignore it |
| "It's stuck for ever" | A loop that doesn't end | CTRL-C. Then ask them which line was supposed to stop it |

## A real Hydra

| WHAT HAPPENS | WHAT TO DO |
|---|---|
| Nothing on the terminal after the power goes on | The port and 9600 baud, first. Then press reset. Then the manual's section 5.7, from the top |
| Nonsense on the terminal | The baud rate: someone changed `/dev/serctl` last lesson, and the terminal program remembers the old speed. The Hydra always starts at 9600 |
| It was working, and now it resets by itself | Something on the I/O header is shorting the supply; or a chip has worked loose. Unplug whatever was added last |
| `/sd/0` isn't there | The card isn't pushed in; the module's wires have come off J18; or it's a new card with no HydraFS (section 14.2) |
| The SD card "lost" a file | The power went off while a program was writing. `fsck 0`, and `fsck -f 0` to repair (section 14.6). Teach: leave the editor before switching off |
| A student has changed something and now it won't start | The settings live on the SD card, in `/sd/0/lib`. Take the card out, and the Hydra starts from its ROM. Then fix the file from another Hydra, or from the PC with `hydrafs.js` |

## The lesson itself

**The projector Hydra fails in front of the class.** It will, one day. Narrate what you do: "the prompt hasn't come back, so first I try CTRL-C..." You're modelling exactly the behaviour you want from them. A teacher calmly recovering from a mistake teaches more than a flawless demonstration.

**Half the class has finished, and half hasn't started.** Stop everyone. Have a finished student show step 1 at the projector. Start again from there, with the finished half on the challenge.

**One student's problem is eating the lesson.** Give it two minutes. If it isn't solved: reset their Hydra, have them start the step again, and write the problem on the board as a puzzle for anyone who finishes early. Most "impossible" problems are a typing slip three steps back, and a clean start is quicker than archaeology.

**Nobody did the reading.** Don't lecture it instead: that teaches them not to read. Give ten minutes of silent reading now, of the two sections that matter most, and cut the lab's last steps.

**The computers are down.** Lab 1 (to step 2) and Lab 24 need only paper. So does planning a project: have them write, by hand, the program they mean to type next lesson. They'll type it faster and it will work sooner.

**A student asks something you can't answer.** Say so, and say "let's find out": the manual's index, then the Hydra itself. With this machine, more than most, the answer really is in there, and looking for it together is the best lesson of the week.



PART THREE

# The Answers

The answers to the Companion's questions, notes on its projects, and a bank of quiz questions.

Where an answer is something the Hydra prints, it's shown as a transcript, and every transcript here was typed into the manual's Hydra and is what it said. Where an answer is in words, it's a model, not a script: mark the understanding, not the wording (Topic 7).


- Topic 10. The Build Log
- Topic 11. Labs 1 to 9
- Topic 12. Labs 10 to 15
- Topic 13. Labs 16 to 23
- Topic 14. Labs 24 to 26
- Topic 15. The Projects
- Topic 16. A Bank of Questions

TOPIC 10

# The Build Log

Most of the build log is a record, not a test: ticks, readings and dates. These are the answers to the questions it does ask, and what to look for in the rest.

## Unit 1. Before You Build

**Which solder, and what does that change?** Lead-free: a hotter iron (about 370 °C), joints that look a little duller even when they're good, and slightly slower wetting. Solder with lead: an easier joint, and stricter hygiene: no food or drink, and hands washed without fail. Either answer is right if it matches what's on the bench.

**Iron temperature.** About 350 °C for tin-lead solder, about 370 °C for lead-free. A student who doesn't know hasn't looked.

**A good joint and a bad one.** Good: a small, shiny cone or volcano, rising smoothly from the pad up the lead, wetting both. Bad: a ball sitting on the pad (the pad wasn't hot); a dull, grainy lump (a cold joint: it moved as it cooled, or wasn't hot enough); a blob joining two pads (a bridge); a hole round the lead (too little solder). Look for a side view, and for *both* the pad and the lead being mentioned.

**Brown, black, orange, gold.** 1, 0, then three zeros: 10,000 ohms, or 10 kΩ. Gold is the tolerance, 5%.

**104, and 330.** Two digits, then a count of zeros, in picofarads. 104 is 10 and four zeros: 100,000 pF, which is 100 nF. 330 is 33 and no zeros: 33 pF. (A student who says 330 pF has read it as a plain number: the commonest mistake.)

**Pin 1.** On a chip: beside the notch at one end, or marked by a dot; pins are counted anticlockwise from there, seen from above. On a socket: at the notched end. On a header: the square pad on the board.

**Parts that must go the right way round.** Electrolytic capacitors (except the two non-polarized ones); every chip and its socket; resistor networks; the reset chip, U43; LEDs; and the keyed connectors (the power connector, by its latch). Resistors, ceramic capacitors and the crystal go either way.

**The kit count** has no right answer but the kit's own. The point is that it's done before the iron is on, and that anything missing is written down.

**The plan.** Look for sessions short enough to be real (one or two hours), a total somewhere near twenty hours, and Step 4 spread over several sessions. Come back to this table at the end: the comparison of "planned" and "took" is one of the more useful things the build teaches.

## Unit 2. Soldering the Main Board

The ticks are the record. The questions:

**Where I stopped (Step 4).** Any note that would let someone else carry on: "sockets done to U24; U26 has one corner tacked".

**What did the inspection find?** Something, always. An empty answer means it wasn't done. Typical finds on a first board: three to ten unsoldered pins (most often on the long sockets and the slots), one or two bridges, a few dull joints to reflow, and now and then a socket the wrong way round. A student who lists what they found, and what they did about each, has done this page properly.

## Unit 3. Power, ROMs and Chips

**The resistance from +5 V to ground.** More than about 100 ohms, usually a few hundred to a few thousand, and climbing as the meter charges the capacitors. Near zero is a short: almost always a solder bridge, sometimes an electrolytic capacitor in backwards.

**The voltages.** +4.75 to +5.25 V on U1's pin 8; about +12 V and −12 V on U42's pins 4 and 11. A missing ±12 V with a good +5 V means the supply's own rails, or the joints at J11: the Hydra will still start without them (they feed only the sound circuit and the slots), which makes it a fault that hides.

**Which programmer, and when were the images built?** A record, for the day the ROMs are programmed again. What matters is that it's filled in.

**DS1747 or HM628512? R65C51 or W65C51N?** Whatever is fitted. It matters later: without a DS1747 the date starts at 2000 every time; and a WDC W65C51N needs HydraOS built for it (the manual's section 4.5).

**The drawing of J10.** Two rows of six, pin 1 at the square pad; the switch across 11 and 9; the reset button across 7 and 5; the LED from 12 (its longer leg) to 10.

## Unit 4. The Memory Card

**How J1 and J2 were kept straight.** The manual's method: push the headers' long pins into the main board's own J1 and J2 sockets, lower the card over their short ends, and solder from the top. Any method works that ends with the pins square to the card, but only this one guarantees they fit the sockets.

**The module numbers.** 0, for a first card. With more than one module, all different.

**What `RAM modules:` will print.** `01`, for one module; in hex, so eleven would be `0B`.

## Unit 5. First Switch-On

**The start-up lines.** For a board that works, with one module:

```
Hydra-16: kernel 0.1, ABI 1
POST ZP:0 ST:0 OS:0 HI:0 SH:S W:0
RAM U:0 F0:0/00/0000 F4:0/00/0000 F8:0/00/0000 FC:0/00/0000 00:0/00/0000
POST ok
RAM modules: 01
```

Zeros everywhere (and `SH:S`) mean no faults. The manual's section 7.4 explains every field, and section 5.7 says which chip each one points at.

**`date`.** `2000-01-01` and a few seconds, without a clock chip.

**`free`.** `128 KB a task (1 module)` for one module, 256 KB for two, 384 KB for three; and `shared 1024 KB, 512 KB in segments (1), 512 KB free`.

**First joint to first prompt, against the plan.** Nearly every build takes longer than planned, mostly in Step 4 and in fault-finding. The good answers say *where* the plan was wrong.

**When it finally worked: what was the fault, and how could you have caught it sooner?** The most valuable answer in the build log. "A chip in backwards; I'd have caught it if I'd checked every notch before the power went on" is the kind of thing to hope for. Read these aloud to next year's class.

**The hardware test.** Every line `ok`, with the CPU clock near 3.58 MHz, the SD card listed if there is one, and "no devices" and "all empty" for the I²C bus and the slots on a bare Hydra. The manual's section 5.7 shows a whole passing run.

TOPIC 11

# Labs 1 to 9

## Lab 1. Meet the Computer

**1. The parts.**

| PART | REFERENCE | ITS JOB |
|---|---|---|
| The processor | U1 | Follows the instructions: everything else serves it |
| The crystal | Y1 | The steady beat that the Hydra's clocks are made from |
| The task RAM | U7 | 32K of memory for each of the sixteen tasks |
| The kernel ROM | U6 | Holds the kernel, the heart of HydraOS |
| The VIA | U2 | Timers (one makes the tick), the SPI bus, and the I/O header's pins |
| The serial chip | U3 | Sends and receives characters, a bit at a time, for the terminal |
| The sound chip | U38 | The YM2151: eight voices of music |
| The socket for the Vera X | J12 (slot 0) | Where the video card connects |
| Where the power comes in | J11 | The ATX power supply's plug |

**2. The conversions.**

| DECIMAL | BINARY | HEX |
|---|---|---|
| 10 | 0000 1010 | `$0A` |
| 65 | 0100 0001 | `$41` |
| 200 | 1100 1000 | `$C8` |
| 16 | 0001 0000 | `$10` |
| 255 | 1111 1111 | `$FF` |
| 42 | 0010 1010 | `$2A` |
| 128 | 1000 0000 | `$80` |

**3. The Hydra's check.** (`calc` doesn't print leading zeros.)

```
/> calc -b b 65
1000001
/> calc -b x 200
C8
/> calc '#x2A'
42
/> calc '#b11111111'
255
```

**Questions.**

1. Input: the serial port (the terminal's keyboard), or the pins of the I/O header. Processing: the W65C02S, U1. Memory: the RAM chips and the ROM chips. Output: the serial port again (the terminal's screen), the sound chip, the Vera X. Accept any sensible part for each.
2. Its sixteen tasks: it can run up to sixteen programs at once, each in memory of its own.
3. 32 × 1,024 = 32,768 bytes. Sixteen of them: 512K (524,288 bytes), which is exactly the size of the task RAM chip, U7.
4. 256, from 0 to 255. Eight bits, each 0 or 1: 2 × 2 × 2 × 2 × 2 × 2 × 2 × 2.
5. RAM can be written as well as read, and forgets everything when the power goes off. ROM keeps what it holds, and a program can't change it. HydraOS is in ROM so that it's there the instant the Hydra is switched on (and so that no program can damage it).
6. Four bits. So a byte is two hex digits, and a 16-bit address is four.

**Challenge.** For example, 2012 is 0111 1101 1100, or `$7DC`. `$FFFF` is 65,535. With five hex digits, `$FFFFF`: 1,048,575, which is one less than a megabyte.

## Lab 2. A Hydra on Your PC

**1.** `HyForth (Forth 2012), bye to end`, and then the prompt, `/>`.

**2.** `5`, and then the twelve names at the top of the file tree, from `bin/` to `tmp/`, each with a `/`.

**3. Pause** stops the Hydra dead: nothing typed appears until it's pressed again. **As fast as it goes** makes everything quicker (the clock at the top right shows how much). The **size** menu changes the console's columns and rows, and the Hydra's windows follow. **Copy text** puts the console's whole text, scrolled-off lines too, on the PC's clipboard. **Sound** turns the sound on (a browser won't make a sound until the user asks).

**4. The experiment.**

| AFTER | /RAM/A.TXT | /SD/0/A.TXT |
|---|---|---|
| Reset | gone | kept |
| Power cycle | gone | kept |
| Closing and reopening the page | gone | kept |

The RAM disk is made afresh every time HydraOS starts, whatever the reason. The SD card is kept by the browser. (Students who report that the SD card's file went too have usually pressed **New** or **Samples** in Setup, or are on a browser that clears its storage.)

**5.** As it comes: a Vera X card in slot 0, without its keyboard and mouse; a clock chip; three RAM modules; the sample SD card in reader 0, and no card in the Vera X's own slot.

**6.** With the Vera X, `ps` has one more line: task 11, `vid`, the screen's driver. Without a Vera X, `vid` starts, finds nothing to drive, and ends.

**Questions.**

1. A program that makes one computer behave like another, by imitating its chips. The same: the processor, the memory, the devices, HydraOS itself, and so every program. Different: no soldering, no real pins or sockets; a perfect "serial cable"; SD cards that are files; and it can run faster than real.
2. On the SD card, `/sd/0`: it's the only place that survived all three.
3. Reset presses the reset button: HydraOS starts again. Power cycle switches off and on: the same, and everything in memory is lost too. (To a student, the visible difference is small: both empty the RAM disks. That's the point.)
4. **Save card**, which gives a file holding the whole card. **Load...** on the other computer brings it back.

**Challenge.** CTRL-A S shows every task and what it's doing. CTRL-A B stops the processor between two instructions; `r` shows its registers at that instant, and `c` lets it carry on.

## Lab 3. First Steps

**1.**

```
/> date
2000-01-01 00:00:04
/> echo Hello, Hydra!
Hello, Hydra!
/> ls /sd/0
bench/
sample/
songs/
/> 2 3 + .
5
/> free
ram     384 KB a task (3 modules)
shared  1024 KB, 512 KB in segments (1), 512 KB free
```

**2.**

```
/> dat
rc: dat: not found
/> LS
rc: LS: not found
/> 2 3 plus .
plus ?
/> 2 3+ .
3+ ?
/> cat /ram/nothing
cat: /ram/nothing: not found
```

| THE MESSAGE | WHICH MEANS |
|---|---|
| `rc: dat: not found` | There's no program called `dat` |
| `rc: LS: not found` | Capitals count: the program is `ls` |
| `plus ?` | The line began with a number, so it's Forth; and Forth has no word `plus` |
| `3+ ?` | Without a space, `3+` is one word, and Forth doesn't know it |
| `cat: /ram/nothing: not found` | The program `cat` ran, and couldn't find that file |

**3.** CTRL-C.

**4.** It prints each line back, as soon as RETURN is pressed. CTRL-D, at the start of a line, ends it.

**5.**

| TO START IT | ITS PROMPT | TO LEAVE IT |
|---|---|---|
| `rc` | `%` | `exit` |
| `basic` | `>` | `SYSTEM` |
| `hylang` | `hylang>` | `(exit)` |
| `: test` | Spaces (no prompt at all) | `;` |

**6.** CTRL-O and then RETURN saved it; CTRL-X left.

**7.** Its mode (who may read and write it), which disk it's on (`f0`: HydraFS, card 0), its length in bytes, when it was last changed, and its name.

**8.** The first time, four tasks each print a line of a song a letter at a time, with nothing to keep them apart, and their letters are shuffled together. With a lock (a mutex) held for a whole line, each line comes out whole, though in whatever order the tasks happened to run. With a baton passed from one to the next, they come out whole and in order.

**Questions.**

1. The program's sign that it's ready for a line. The Hydra's also shows the current directory.
2. Just `two`: `>` starts the file afresh each time. With `>>`, both lines: `one`, then `two`.

3. It's made of memory, and is emptied at every reset and power-off. `/sd/0`, the SD card (or `/pc`, with no card reader).
4. The name is misspelled; a capital letter where a small one should be; the file is in another directory (so it needs its path); or the first word was meant as Forth and isn't a Forth word.
5. Chapter 11 of the manual, under W: the commands are in alphabetical order.

**Challenge.** For example:

```
/> : me ." My name is Sam" cr ;
/> me
My name is Sam
```

```
> 10 FOR I = 1 TO 5
> 20 PRINT "Sam"
> 30 NEXT I
> RUN
Sam
Sam
Sam
Sam
Sam
```

```
hylang> (print "Sam")
Sam
=> NIL
```

## Lab 4. From Power to Prompt

**1.**

```
/> cat /dev/kmesg | head -6

Hydra-16: kernel 0.1, ABI 1
POST ZP:0 ST:0 OS:0 HI:0 SH:S W:0
RAM U:0 F0:0/00/0000 F4:0/00/0000 F8:0/00/0000 FC:0/00/0000 00:0/00/0000 10:0/00/0000
20:0/00/0000
POST ok
RAM modules: 03
```

**2.** (The task numbers, and `ps`'s own, may differ.)

```
/> ps
task  state   parent     cpu group  name
   0  ready   -          1.6     0  kernel
   1  wait    -          0.4     1  forth
   2  ready    1         0.0     1  ps
  12  idle    -          0.0    12  snd
  13  idle    -          0.1    13  kdev
  14  idle    -          1.4    14  storage
  15  event   -          0.5    15  cons
```

| TASK | NAME | WHAT IT'S FOR |
|---|---|---|
| 0 | `kernel` | The kernel's own task: it also runs when nothing else can |
| 1 | `forth` | The shell you're typing at. (`init` set the system up in this task, then became the shell: no task is kept only to wait) |
| 2 | `ps` | The command itself |
| 12 to 15 | `snd`, `kdev`, `storage`, `cons` | The drivers: the sound, the kernel's devices (and the pins), the disks, the console |

**3.** `ps` is a program, so while it runs, it's a task: it lists itself. When it finishes, its task is free again, for the next program that starts.

**4.**

```
/> sleep 60 &
/> ps
task  state   parent     cpu group  name
   0  ready   -          2.1     0  kernel
   1  wait    -          0.4     1  forth
   2  sleep    1         0.0     2  sleep
   3  ready    1         0.0     1  ps
  12  idle    -          0.0    12  snd
  13  idle    -          0.1    13  kdev
  14  idle    -          1.5    14  storage
  15  event   -          0.5    15  cons
/> kill 2
/> ps
task  state   parent     cpu group  name
   0  ready   -          3.9     0  kernel
   1  wait    -          0.4     1  forth
   3  ready    1         0.0     1  ps
  12  idle    -          0.0    12  snd
  13  idle    -          0.1    13  kdev
  14  idle    -          1.6    14  storage
  15  event   -          0.6    15  cons
```

`sleep` was in the state `sleep`, with the shell (task 1) as its parent. After `kill`, it's gone from the list.

**5.** 384 KB for each task, on the manual's Hydra: three RAM modules, at 128 KB each.

```
/> free
ram     384 KB a task (3 modules)
shared  1024 KB, 512 KB in segments (1), 512 KB free
```

**6.** Two numbers a few hundred apart: there are 200 ticks in a second.

**Questions.**

1. Through "windows": parts of the 64K that show a piece of a bigger memory, with a register choosing which piece. `$8000` to `$9FFF` shows one 8K RAM bank at a time, and `$A000` to `$DFFF` one 16K bank of ROM.
2. A program that's running, with memory of its own. The T register chooses which task's 32K the processor sees, so a task simply can't address another task's memory; and only the kernel changes T.
3. The processor reads the reset vector; the power-on self test; the drivers start; `init` starts; the shell starts.
4. A program that looks after a device, so that other programs don't touch the chips themselves. `cons`: the serial port and the windows. `storage`: the SD cards and the other disks. `snd`: the sound chip. (Also `kdev`: the clock, the tasks, the environment and the I/O header; `vid`: the Vera X.)
5. Another starts at once, so there's always a shell to type at: the kernel starts `init` again, and `init` becomes the new shell. Without that, one mistyped `bye` would leave the Hydra with no way in.
6. A timer in the VIA interrupts the processor 200 times a second. On each tick, the kernel can stop the task that's running and give another its turn, and it counts the time.

**Challenge.** `mods | wc -l` says 16: a heading, and fifteen modules. `forth` is in 1 bank (bank 30), `basic` in 5 (banks 9 to 13), and `hylang` in 8 (banks 14 to 21). hylang is by far the biggest language to *implement*, and Forth the smallest: one reason Forth was so popular on small machines.

## Lab 5. Talking to the Shell

**1.**

```
% pets=(cat dog fish)
% echo $pets
cat dog fish
% echo $#pets
3
% echo $pets(2)
dog
% echo I have a $pets(1)
I have a cat
% echo 'I have a $pets(1)'
I have a $pets(1)
```

**2.** `3`. The < makes `wc` read the file as its input, as if it had been typed.

```
% cat /ram/n.txt
one
two
three
% wc -l </ram/n.txt
      3
```

**3.** 47: the programs on the ROM disk. 6: the names in `/sd/0/sample`. And the three words, in alphabetical order.

```
% ls /rom/bin | wc -l
     45
% ls /sd/0/sample | wc -l
      6
% cat /ram/n.txt | sort
one
three
two
```

**4.** Two words: the date, and the time.

```
% echo today is `{date}
today is 2000-01-01 00:00:04
% now=`{date}
% echo $#now
2
```

**5.** Empty after the `ls` that worked, and `1` after the one that failed. `||` runs the second command only if the first failed; `&&` only if it worked.

```
% ls /rom >/dev/null
% echo $status

% ls /nowhere
ls: /nowhere: not found
% echo $status
1
% ls /nowhere >[2]/dev/null || echo not there
not there
% ls /rom >/dev/null && echo there
there
```

**6.**

```
% ls /rom/bin/s*
/rom/bin/scom
/rom/bin/slay
/rom/bin/sleep
/rom/bin/sort
% ls /rom/bin/[x-z]*
/rom/bin/xd
/rom/bin/xmodem
```

`/rom/bin/??` lists the thirteen programs with two-letter names: `as`, `cp`, `db`, `df`, `du`, `ed`, `ls`, `mv`, `ns`, `ps`, `rm`, `wc` and `xd`.

**Questions.**

1. By the line's first word: if it's a Forth word that HyForth knows, or a number, the line is Forth; otherwise it's rc's. A `%` and a space at the start sends a line to rc whatever it begins with; or type `rc` and work there.
2. The first prints the variable's words; the second prints the six characters `$pets`. Quotes are for when a word has spaces or special characters in it that the shell should leave alone.
3. To `/dev/null`, a file that throws away whatever is written to it. `2` is the number of the **standard error** file: programs write their error messages there, and their results to file 1, so that the two can be sent to different places.
4. It looks in each directory in `$path`, in order: first `.`, the current directory, then `/bin`. `whatis ls` says `/bin/ls`.
5. `ls /rom/bin/s* | wc -l`, or `ls /rom/bin | grep '^s' | wc -l`. Both say 4.

**Challenge.**

```
% ls /rom/bin | sort -r | head -3
xmodem
xd
wc
% ls /rom/bin | grep e | wc -l
     15
```

Nine, because each program in a pipeline is a task: the Hydra has sixteen, and seven are in use at rc's prompt (the kernel, four drivers, HyForth and rc: `ps` shows them, and itself), which leaves nine. For a job that needs ten programs, send the half-way result to a file in `/ram`, and start a second pipeline from that file.

## Lab 6. Scripts

**1.**

```
% for(i in 1 2 3) echo number $i
number 1
number 2
number 3
% for(f in /sd/0/sample/*) echo I found $f
I found /sd/0/sample/as
I found /sd/0/sample/c
I found /sd/0/sample/hi
I found /sd/0/sample/nsum
I found /sd/0/sample/tick
I found /sd/0/sample/upper
```

**2.** The first, the second and the fourth. `Sam` matches `Sam`, and it matches `S*`. It doesn't match `Ann`, so that `if` prints nothing, and the `if not` that follows it does.

```
% name=Sam
% if(~ $name Sam) echo hello Sam
hello Sam
% if(~ $name S*) echo starts with S
starts with S
% if(~ $name Ann) echo hello Ann
% if not echo you are not Ann
you are not Ann
```

**3.**

```
% fn shout { for(w in $*) echo $w! }
% shout one two three
one!
two!
three!
```

**4.** `#!/bin/rc` says which program should run the file: rc. The `#` line is a comment, for people.

```
/ram> rc /ram/tour
In /rom there are:
README
bin/
doc/
lib/
songs/
In /ram there are:
bin/
lib/
tour
In /sd/0 there are:
bench/
sample/
songs/
```

**5.** Into `/ram/bin`. `/bin` is a union, and new files made in a union go into the first of its directories that allows it: the RAM disk's `bin`. (`ls /sd/0/bin` says `not found`: the sample card has no `bin` directory.)

**6.** One way:

```
#!/bin/rc
# tour - say what's on each disk, or in each directory named
dirs=$*
if(~ $#dirs 0) dirs=(/rom /ram /sd/0)
for(d in $dirs) {
        echo In $d there are:
        ls $d
}
```

**Questions.**

1. A function lives in the shell's memory, and is gone when that shell ends. A script is a file: it's there tomorrow, and any shell can run it. A function for something you want for the next ten minutes; a script for anything you want to keep.
2. The shell. It replaces the pattern with the list of matching names *before* `for` or any command sees it.
3. It's lost, with the rest of the RAM disk. A script to keep goes in `/sd/0/bin` (make the directory; it joins `/bin` the next time the namespace is built: after a reset, or the command `newns`).
4. "While it is not true that `$i` matches `0`": `~` tests a match, `!` turns success into failure and back. So: carry on until `$i` is 0.

**Challenge.** One way:

```
#!/bin/rc
# countdown - count down from a number, a second at a time
i=$1
while(! ~ $i 0) {
        echo $i
        sleep 1
        i=`{calc $i - 1}
}
echo Lift off!
```

## Lab 7. The Tree of Files

**1.**

```
/> cd /rom/lib/forth
/rom/lib/forth> cd ..
/rom/lib> cd ../songs
/rom/songs> cd /
```

**2.** `/` at the top; `rom` under it; `lib` and `songs` side by side under `rom`; `forth` under `lib`.

**3.** A directory's line starts with `d`. The only file there is `README`, of 1,955 bytes; the directories show a length of 0.

```
/> ls -l /rom
--r--r--r-- fx     1955 2000-01-01 00:00 README
d-r--r--r-- fx        0 2000-01-01 00:00 bin
d-r--r--r-- fx        0 2000-01-01 00:00 doc
d-r--r--r-- fx        0 2000-01-01 00:00 lib
d-r--r--r-- fx        0 2000-01-01 00:00 songs
```

**4.**

```
/> df
disk  kind   size        free        label
0     sdhc   16380 KB    15956 KB    HYDRA-16
x     rom    623 KB      11 KB       ROM
r     ram    255 KB      253 KB      RAM
s     sram   511 KB      510 KB      SRAM
```

Disk `0` is the SD card, at `/sd/0`; `x` the ROM disk, at `/rom`; `r` the RAM disk, at `/ram`; `s` the shared RAM disk, at `/sram`.

**5.** The date and time; a count of ticks, which is different each time; and 28 names (the manual's Hydra has seven directories and twenty-one files in `/dev`).

**6.** Nothing; then `scom.zsm`; then nothing again. Nothing was copied: for a while, `/mnt` was another *name* for the directory `/rom/songs`.

```
/> ls /mnt
/> bind /rom/songs /mnt
/> ls /mnt
scom.zsm
/> unmount /mnt
/> ls /mnt
```

**7.** Four lines: so `/bin` is four directories seen as one.

```
bind -c '#fr/2/bin' /bin
bind -a '#fs/bin' /bin
bind -a '#fx/bin' /bin
bind -a '#m/bin' /bin
```

They are the RAM disk's `bin` (where new files go: that's the `-c`), the shared RAM disk's, the ROM disk's, and the programs in the paged ROM itself. A card with a `bin` directory adds a fifth.

### Questions.

1. The list of directories that leads to a file from the root. `/ram/notes` starts at the root, so it means the same thing everywhere; `notes` starts from the current directory, so it depends on where you are.
2. `.` is the directory you're in; `..` is the one above it.
3. `/rom`, `/ram`, `/sram`, and `/sd/0` (and `/sd/1` and so on, for more cards).
4. Any two of the four above. The one that comes first in the union: the directories are searched in order.
5. No. A namespace belongs to a task (and the tasks it starts). Section 9.5 does the experiment: a `bind` made inside an `rc` isn't there after `exit`.

**Challenge.** Yes. `-a` adds the new directory *after* the ones already in the union, so it's searched last; `-b` would put it *before*, so a script of yours with the same name as a command would win.

## Lab 8. Windows

**1.** A list of the window keys. Any key puts it away.

**2.** The screen clears, and a new shell says hello: a new window, in a new group. It's window 1, and the bar at the top shows it.

**3.** 0: window 0, just as it was left. 1: the second window, with its line still there. W: a list of the windows, each with its number, its group and the program running in it. N: the next group's window.

**4.** Yes: `ps`, typed in the second window, lists `sleep`. A window isn't a program; it's a screen and a keyboard for one. The programs in the windows you aren't looking at carry on.

**5.** The screen is split into two columns, one window in each. CTRL-] and an arrow key moves the keyboard between them.

**6.** `columns` or `rows`: the editor in one, and a shell in the other.

**7.** In the window's heading, at the top.

**8.** The shell ends, and at once a new one starts in the same window: the kernel starts `init` again, which becomes the new shell. Window 0 never goes away.

**Questions.**

1. Each window has its own screen, kept by the console even while it isn't shown, and its own shell. You choose which one the terminal shows, and which gets the keys.
2. A window is one screen with one program at its keyboard. A group is a set of windows that belong to one shell session, and can be shown together, tiled.
3. Press CTRL-] twice.
4. A file served by the console driver, which stands for this window's label. Likewise `/dev/cons`, the window's keyboard and screen; `/dev/consctl`, its settings; and `/dev/wctl`, its group's layout.

## Lab 9. A Command Hunt

**1.** `wc`; `head`; `tail`; `grep`; `sort`; `more`; `xd`; `cmp`; `df`; `top`; `slay`; `sleep`.

**2.**

```
/> wc -l /sd/0/sample/as/hi.s
     84 /sd/0/sample/as/hi.s
/> tail -1 /sd/0/sample/as/hi.s
s_end:      .byte       ".", LF, 0
/> grep -c PRINT /sd/0/sample/as/hi.s
9
```

`head -3` shows its first three lines, which are comments; and `grep -n window` finds the word on lines 3, 8, 21, 64, 65, 73, 74, 75 and 83.

**3.**

```
/> sort /ram/fruit
apple
fig
pear
pear
/> sort -r /ram/fruit
pear
pear
fig
apple
/> sort -u /ram/fruit
apple
fig
pear
/> wc -c /ram/fruit
     20 /ram/fruit
/> xd /ram/fruit
0000000  70 65 61 72 0a 61 70 70 6c 65 0a 70 65 61 72 0a  pear.apple.pear.
0000010  66 69 67 0a                                      fig.
```

(`sort /ram/fruit | uniq` does the same as `sort -u`.)

**4.** `p` is `$70`. Each line ends with `$0A`, the new-line character.

**5.** Square brackets mean "you can leave this out". `...` means "as many as you like". The pattern must always be given.

**Questions.**

1. A command built into the shell. The current directory belongs to a task. A separate program would be a separate task: it would change its *own* directory, and then end, leaving the shell's where it was.
2. `wc`, `sort`, `grep`, `head` and `uniq` all do.
3. `kill` takes a task's number; `slay` takes a program's name, and stops every task running it.
4. Any three, with reasons. Expect `ls`, `cat`, `edit`, `cp` and `ps`.

**Challenge.** `d` and `m`, with five each (`date`, `db`, `df`, `dis`, `du`; and `mkdir`, `mkfs`, `mods`, `more`, `mv`). One way, trying the likely letters:

```
% for(l in c d m s t) echo $l `{ls /rom/bin/$l^* | wc -l}
c 4
d 5
m 5
s 4
t 4
```

The `^` matters: `$l*` would be read as a variable called `l*`, which is empty (rc says `null list in concatenation`). `$l^*` joins the letter to the `*` (section 8.5).

TOPIC 12

# Labs 10 to 15

## Lab 10. Devices Are Files

**1.** Any four, matched to their drivers by the table in section 12.3. For example: `cons`, `consctl`, `wctl` and `label` are the console's; `snd`, `sndctl`, `psg` and `bell` the sound driver's; `ticks`, `time`, `rtc`, `null`, `zero` and `kmesg` the kernel's own; and the directories `gpio`, `i2c`, `spi`, `sd` and `vid` are the pins, the I²C bus, the SPI devices, the disks and the Vera X's screen.

**2.**

```
/> cat /dev/serctl
b9600
x1
pc 0 req, 0 cut short, 0 unanswered, 0 damaged, 0 nak, 0 gone
line 0 overruns, 0 framing errors, 0 dropped
/> cat /dev/consctl
rawoff
keys hydra
scroll smooth
group 1
window 0
size 80 24
terminal serial
/> cat /dev/sd/0/ctl
sdhc 17 MB 34816 blocks
hydrafs label=HYDRA-16
partition at block 2048
free 15956 KB of 16380 KB
/> cat /dev/time
2000-01-01 00:00:07
```

**3.** `echo` said it, because it was `echo` whose write failed; but the *reason*, `invalid argument`, came back from the driver, which didn't know the command `fly`.

```
/> echo fly >/dev/gpio/ctl
echo: write error: invalid argument
```

**4.** `1`, and then `0`.

```
/> echo out 2 >/dev/gpio/ctl
/> echo 1 >/dev/gpio/2
/> cat /dev/gpio/2
1
/> echo 0 >/dev/gpio/2
/> cat /dev/gpio/2
0
```

**5.** The words `red`, `green` and `blue`, each in its own colour. `ESC [0m` puts the colours and styles back to normal. Without it, everything printed afterwards (the prompt, and what's typed) stays blue.

**6.**

| THE JOB | THE SEQUENCE |
|---|---|
| Clear the whole screen | `ESC [2J` (and `ESC [H` to put the cursor at the top left) |
| Row 5, column 10 | `ESC [5;10H` |
| Bright green text | `ESC [92m` |
| Reversed text | `ESC [7m` |

**Questions.**

1. It knows how to work a device's chips, and offers the device to every other program as a few files. If each program talked to the chips itself, each would need to know every chip, and two programs using a device at once would trample each other. The driver is the one place that knows, and it takes turns.
2. Reading it shows the device's settings; writing a line of text to it gives the device a command.
3. For example: `cat /dev/time` reads the clock with the program made for showing files; `echo` sets a pin, or the serial port's speed; `ls` shows which disks there are.
4. `invalid argument`: the driver didn't understand the command, or a number was out of range. `busy`: someone else is using the device. `no such device`: it isn't there (no Vera X, no card).
5. The terminal program on the PC has to be set to the same speed. The two ends of a serial line share no clock: each just counts time from the start of every character, so they must agree in advance how long a bit lasts.

**Challenge.** In HyForth, with the `esc` from step 5:

```
: banner ( -- ) esc ." [2J" esc ." [12;34H" esc ." [1;33;44m  Sam  " esc ." [0m" cr ;
```

In rc, the hard part is typing an ESC into the script: the editor takes the key for itself. The manual's entry for `echo`, and section 13.7, give the ways round it.

## Lab 11. Disks

**1.**

```
/> ls /dev/sd
0/
x/
r/
s/
/> cat /dev/sd/0/ctl
sdhc 17 MB 34816 blocks
hydrafs label=HYDRA-16
partition at block 2048
free 15956 KB of 16380 KB
/> cat /dev/sd/x/ctl
rom 4 MB 8192 blocks
hydrafs label=ROM
partition at block 1024
free 11 KB of 623 KB
/> cat /dev/sd/r/ctl
ram 256 KB 512 blocks
hydrafs label=RAM
free 253 KB of 255 KB
banks $00-$1f
/> cat /dev/sd/s/ctl
sram 512 KB 1024 blocks
hydrafs label=SRAM
free 510 KB of 511 KB
segment 15
```

| DISK | WHAT KIND | ITS SIZE | SEEN AT |
|---|---|---|---|
| `0` | An SD card (`sdhc`) | 17 MB, on the manual's Hydra | `/sd/0` |
| `x` | The ROM disk | 623 KB of files, in the 4 MB of paged ROM | `/rom` |
| `r` | A RAM disk, in RAM banks | 256 KB | `/ram` |
| `s` | A RAM disk, in a shared segment | 512 KB | `/sram` |

**2.** `read-only`. The ROM disk is in ROM chips, which a running Hydra can't write.

```
/> echo hello >/rom/test
rc: /rom/test: read-only
```

**3.** Two bytes: the `x`, and the new line. But the free space fell by 4 KB. A disk hands out space in whole **clusters**, and on a card a cluster is 4K: the smallest file takes one, and a file of 4,097 bytes takes two.

```
/> df
disk  kind   size        free        label
0     sdhc   16380 KB    15956 KB    HYDRA-16
x     rom    623 KB      11 KB       ROM
r     ram    255 KB      253 KB      RAM
s     sram   511 KB      510 KB      SRAM
/> echo x >/sd/0/tiny
/> ls -l /sd/0/tiny
--rw-rw-rw- f0        2 2000-01-01 00:00 /sd/0/tiny
/> df
disk  kind   size        free        label
0     sdhc   16380 KB    15952 KB    HYDRA-16
x     rom    623 KB      11 KB       ROM
r     ram    255 KB      253 KB      RAM
s     sram   511 KB      510 KB      SRAM
```

**4.** No faults: nothing lost, nothing unmarked, nothing used twice.

```
/> fsck s
hydrafs label=SRAM
free 510 KB of 511 KB
check: lost 0, unmarked 0, twice 0
segment 15
```

**5.** In kilobytes:

```
/> du /sd/0/sample
14      /sd/0/sample/as
188     /sd/0/sample/c
208     /sd/0/sample
/> du /rom/bin
224     /rom/bin
```

**6.** `HYDRA-16`, on the sample card.

### Questions.

1. SD cards: big, kept when the power's off, and they can be carried to a PC; the place for anything that matters. RAM disks: very fast, and emptied at every start; a scratch pad. The ROM disk: HydraOS's own files, always there and never changed.
2. A block is 512 bytes: a disk is read and written a block at a time. A cluster is the unit that space is handed out in: 8 blocks (4K) on a card, 2 blocks (1K) on the RAM disks and the ROM disk.
3. An SD card module: a small board with a voltage regulator and a level shifter.
4. A disk is only a long row of blocks. A file system is the agreement about where, in that row, the names, the lengths and the contents go. A new card must be given an empty HydraFS first: `mkfs 0`.
5. Any two of: `/pc`, with the PC tool; `xmodem`; taking the card to the PC and using `hydrafs.js` on it or on its image; and, on the emulator, a card image or `--pc-dir`.

**Challenge.** The shared RAM disk, `s`, can be stopped and started again at another size. `start` takes a number of 8K banks, so 16 makes a disk of 128 KB, and 64 puts back the usual 512 KB. Its memory is a shared segment: `free` shows the segments' share fall to nothing at `stop`, and come back at `start`.

```
/> echo stop >/dev/sd/s/ctl
/> free
ram     384 KB a task (3 modules)
shared  1024 KB, 0 KB in segments (0), 1024 KB free
/> echo start 16 >/dev/sd/s/ctl
/> df
disk  kind   size        free        label
0     sdhc   16380 KB    15956 KB    HYDRA-16
x     rom    623 KB      11 KB       ROM
r     ram    255 KB      253 KB      RAM
s     sram   127 KB      127 KB      SRAM
/> free
ram     384 KB a task (3 modules)
shared  1024 KB, 128 KB in segments (1), 896 KB free
```

## Lab 12. Sound

**1.** `patch 0 0` gave channel 0 the first instrument, a piano. `note 0 60` started note 60, middle C, on channel 0. `off 0` let it go. The two numbers after `note` are the channel and the note.

**2.** D: 62. The A of 440 Hz: 69. The C above: 72. The C below: 48.

**3.** Any five. Section 15.4 has the list; the numbers are General MIDI's, counted from 0.

**4.** `o4` chooses octave 4, where `c` is middle C. `l8` makes notes eighth notes unless they say otherwise. `>` goes up an octave, so the last `c` is the C above. The third line played three notes together, C, E and G, one on each of channels 0, 1 and 2: a chord of C major.

**6.** `A` and `B` are the channels the lines belong to: channel 0 and channel 1. `#tempo` is the speed, in quarter notes a minute.

**Questions.**

1. Play eight notes at once: chords, or several instruments together.
2. A patch is an instrument: a set of settings for the chip that makes one kind of sound. A channel is one of the chip's voices: it plays one note at a time, with one patch.
3. 440 vibrations a second. 880 Hz is the A an octave higher: doubling the frequency goes up exactly one octave. (Note 81.)
4. A ZSM file is the list of values to write to the sound chips' registers, and when: all the working-out already done. A score is for people to write and read; a ZSM file is for the Hydra to play, quickly, and it's what other music programs can produce. `play -o` turns the first into the second.
5. Faster than real time, the Hydra's clock runs fast, so every note is short and the tune races.

**Challenge.** For example, a bass drum on every beat, on a third channel:

```
C x36 x36 x36 x36 | x36 x36 x36 x36 | x36 x36 x36 x36 | x36 x36 x36 x36
```

The ZSM file is the bigger of the two: it spells out every register write that the score only describes.

## Lab 13. Pictures

**1.** The console: the same text as the serial terminal, in 80 columns and 60 rows, with a bar at the top (the groups, and the time) and the window's heading. It's different in being a screen of the Hydra's own, with colours and a font the Hydra chooses.

**2.** On a Hydra with a Vera X:

```
vera 47.0.2
mode 80x60
cursor blink
border 0
bitmap off
output vga
```

For example, `mode 80x60` is the size of the text, and `bitmap off` says there's no picture under it yet.

**4.** (0, 0) is the top left corner. (319, 239) is the bottom right. *y* goes *down* the screen.

**5.** One house, of many:

```
clear 0
pen 14
box 100 120 220 200
line 100 120 160 70
line 160 70 220 120
pen 9
bar 150 160 170 200
pen 11
disc 190 145 10
```

**6.** `draw` takes one command on each line it's sent, and doesn't care who sends it. `cat` just copies the file's lines to the device, exactly as `echo` sent them one at a time.

**Questions.**

1. A screen for a monitor; sprites; more sound (the PSG's sixteen voices, and recorded sound); an SD card socket; and, with its controller, a keyboard and a mouse.
2. One dot of the picture, with a colour of its own. 320 × 240 = 76,800.

3.256. Four bits: 16.

4. `box` and `circle` draw outlines; `bar` and `disc` are filled.
5. A drawing that's text can be kept in a file, changed with the editor, made by a script or by a program in any language, and sent from somewhere else. None of that needs a drawing program to exist.

**Challenge.** One way. `calc`'s sum is quoted, to keep the `*` from the shell.

```
#!/bin/rc
echo bitmap 320 8 >/dev/vid/ctl
echo clear 0 >/dev/vid/draw
for(i in 1 2 3 4 5 6 7 8 9 10) {
        echo pen $i >/dev/vid/draw
        echo circle 160 120 `{calc $i '*' 10} >/dev/vid/draw
}
```

## Lab 14. Pins, and the System's Own Files

**1.** A line for each of the eight pins: its number, whether it's an input or an output, and its level; then the two handshake lines, CA1 and CA2.

```
/> echo out 2 >/dev/gpio/ctl
/> echo 1 >/dev/gpio/2
/> cat /dev/gpio/2
1
/> echo 0 >/dev/gpio/2
/> cat /dev/gpio/ctl
0 in 1
1 in 1
2 out 0
3 in 1
4 in 1
5 in 1
6 in 1
7 in 1
ca1 fall 0
ca2 in
```

**2.** PA2 is pin 5 of J27. +5 V is pin 2, and ground is pin 1.

**3.** From pin 2 (+5 V), through the resistor, into the LED's longer leg; out of its shorter leg, to pin 5 (PA2).

**4.** Writing 0 lights it. Current flows from +5 V, through the LED, *into* the pin, so the pin has to be low. It's wired this way round because a VIA pin can take in much more current than it can give out.

**5.** CTRL-C stops it. A smaller number after each `sleep` makes it faster.

**6.** On the manual's Hydra:

```
/> cat /proc/1/status
forth wait - 89 1
/> cat /proc/1/fd
/
0 rw #c 367 #c/cons
1 rw #c 367 #c/cons
2 rw #c 367 #c/cons
```

`status` is the task's name, its state, its parent, the processor time it's had (in ticks), and its note group. `fd` starts with its current directory, and then lists its open files: for a shell, the window's keyboard and screen, as files 0, 1 and 2.

**7.**

```
/> ls /env
prompt
path
task
prog
/> cat /env/path
./bin/>
```

(`cat` printed `/env/path`'s value with the next prompt stuck to its end: the value has no new line of its own, and `cat /env/path; echo` adds one.) `/env/prompt` holds the two prompts: the usual one, and the one shown when a command goes on to another line.

**8.** `none`, on a Hydra with no clock chip: it will have forgotten by tomorrow, and start again at the year 2000.

```
/> echo 2026-10-12 09:41:00 >/dev/time
/> date
2026-10-12 09:41:01
/> cat /dev/rtc
none
```

**Questions.**

1. An input is read by the Hydra; an output is set by it. A switch or a sensor goes to an input; an LED or (through a transistor) a motor to an output.
2. To limit the current. Without one, far too much flows, and the LED or the pin is damaged.
3. They're the I²C bus's two wires, SCL and SDA. While a program is using the bus, the pin commands for them don't work; and anything wired to them as ordinary pins would upset the bus.
4. The kernel makes them up, from what it knows about each task, at the moment they're read. `ps` reads them; and so does `db`, the debugger, which is an ordinary program that works entirely through `/proc`.
5. One set of tools works on everything. The commands for files (`cat`, `echo`, `ls`) and every language's file words can reach every device, with nothing special to learn for each, and a new device needs no new commands.

**Challenge.** One way:

```
% echo out 2 >/dev/gpio/ctl; echo in 3 >/dev/gpio/ctl
% while(~ 1 1) { if(~ `{cat /dev/gpio/3} 0) echo 0 >/dev/gpio/2; if not echo 1
>/dev/gpio/2 }
```

Without a real Hydra, the task names:

```
% for(t in /proc/*) cat $t/status
kernel ready - 480 0
forth wait - 87 1
snd idle - 4 12
kdev ready - 42 13
storage idle - 296 14
cons event - 103 15
rc wait 1 30 1
```

## Lab 15. Numbers and Bases

**1.**

```
/> calc '2^10'
1024
/> calc '2^64'
18446744073709551616
/> calc '10 / 4'
5/2
/> calc '1/3 + 1/6'
1/2
/> calc '0.1 + 0.2'
0.3
/> calc '1/3 * 3'
1
/> calc 'sqrt 16'
4
/> calc 'sqrt 2'
1.41421356237
/> calc 'sqrt(-4)'
2i
```

**2.** It keeps fractions as fractions, exactly, instead of turning them into decimals that have to be cut off somewhere; and it keeps decimals in base ten, not in binary, so 0.1 really is a tenth.

**3.**

```
/> calc -d 30 'sqrt 2'
1.41421356237309504880168872421
```

**4.**

| DECIMAL | BINARY | HEX | OCTAL |
|---|---|---|---|
| 10 | 1010 | A | 12 |
| 100 | 1100100 | 64 | 144 |
| 255 | 11111111 | FF | 377 |
| 1000 | 1111101000 | 3E8 | 1750 |

**5.**

```
/> calc '#b1010'
10
/> calc '#xFF'
255
/> calc '#b1010 + #x0F'
25
/> calc -b x '#xFF + 1'
100
```

**6.** Base 3 (ternary), with the digits 0, 1 and 2; and base 36, which uses all ten digits and all 26 letters.

```
/> calc -b t 10
101
/> calc -b z 1000000
LFLS
```

**Questions.**

1. Whole numbers of any size (`2^64`); fractions (`1/3`); decimals (`0.1`); and complex numbers (`2i`).
2. The Hydra keeps the third as the fraction 1/3, and three of those are exactly 1. A calculator keeps 0.33333333, which is a little less than a third, and three of those are 0.99999999.
3. Eight digits, 0 to 7. The biggest is 7.
4. `A` is `1010` and `5` is `0101`, so `$A5` is `1010 0101`.
5. `division by zero`, or words to that effect. A program should check before it divides, or catch the error (each language's chapter shows how).

```
/> calc 1/0
calc: division by zero
```

**Challenge.** 302 digits. `wc -c` says one more, because it counts the new line at the end.

```
/> calc '2^1000' | wc -c
    303
```

In balanced ternary, written least digit first: 5 is `--+` (−1 − 3 + 9), 6 is `0-+` (0 − 3 + 9), and 7 is `+-+` (1 − 3 + 9).

```
/> calc -b c 5
--+
/> calc -b c 6
0-+
/> calc -b c 7
+-+
```

TOPIC 13

# Labs 16 to 23

## Lab 16. HyForth: The Stack

**1.** (Each line is followed here by enough `drop`s to clear the stack.)

```
/> 1 2 3 .s
<3> 1 2 3
/> drop drop drop 1 2 3 swap .s
<3> 1 3 2
/> drop drop drop 1 2 3 rot .s
<3> 2 3 1
/> drop drop drop 1 2 over .s
<3> 1 2 1
/> drop drop drop 5 dup * .s
<1> 25
/> drop 10 2 3 + * .s
<1> 50
/> drop 10 2 * 3 + .s
<1> 23
```

**2.**

```
/> 4 5 + .
9
/> 4 5 + 2 * .
18
/> 100 7 8 * - .
44
/> 12 4 + 9 5 - / .
4
/> 100 7 mod .
2
```

**3.** It's a comment: a **stack diagram**, saying that `square` takes one number and leaves one. HyForth skips everything between `(` and `)`. It's for the reader.

```
/> : square ( n -- n*n ) dup * ;
/> 9 square .
81
/> see square
: square dup * ;
```

**4.** For example:

```
/> : double ( n -- 2n ) 2 * ;
/> : cube ( n -- n^3 ) dup dup * * ;
/> : average ( a b -- avg ) + 2 / ;
/> : hello ( -- ) ." Hello from Sam" cr ;
/> 21 double . 3 cube . 10 20 average .
42 27 15
/> hello
Hello from Sam
```

**5.** The prompt becomes a row of spaces: HyForth is still inside the definition, compiling. A `;` ends it.

**6.**

```
/> drop
stack underflow
/> 1 0 /
division by zero
```

**Questions.**

1. A pile of numbers. A number typed goes on the top, and words take their numbers from the top and put their answers back there: the last one on is the first one off, like a stack of plates.
2. The order of the words *is* the order things are done in, so no brackets are ever needed, and no rules about which operation comes first. (And it makes the language tiny: every word works the same way.)
3. `.` takes the top number off the stack and prints it. `.s` shows the whole stack, and leaves it alone.
4. There's no space before the `;`. Forth reads `2*;` as one word, doesn't know it, and gives up on the definition:

```
/> : double 2*;
2*; ?
```

5. 32,767 (or 65,535, if it's read as unsigned). `32767 1 + .` prints `-32768`: the sum has gone past the largest positive number a 16-bit cell can hold, and wrapped round to the most negative.

**Challenge.**

```
/> : f>c ( f -- c ) 32 - 5 * 9 / ;
/> 212 f>c . 32 f>c . 98 f>c .
100 0 36
/> : c>f ( c -- f ) 9 * 5 / 32 + ;
/> 100 c>f . 37 c>f .
212 98
```

98 °F is 36.67 °C, and `/` throws the remainder away. To keep one more digit, work in tenths: multiply by 50, not by 5, and read the answer 366 as 36.6.

## Lab 17. HyForth: Decisions, Loops and Programs

**1.** `-1`, `0`, `-1`, `-1`. True is -1 (every bit a 1), and false is 0.

```
/> 5 3 > . 5 3 < . 7 7 = . 0 0= .
-1 0 -1 -1
```

**2.** `0<` uses up the number it tests, so `dup` makes a copy for it. That leaves the original on the stack, and the `drop` at the end throws it away, to keep the promise of the diagram ( *n* -- ). Without the `drop`, every call would leave a number behind.

```
/> : sign ( n -- ) dup 0< if ." negative" else ." not negative" then drop cr ;
/> 5 sign
not negative
/> -5 sign
negative
/> 0 sign
not negative
```

**3.**

```
/> : big? ( n -- ) 100 > if ." big" else ." small" then cr ;
/> 500 big?
big
/> 5 big?
small
```

**4.**

```
/> : a 5 0 do i . loop ;
/> a
0 1 2 3 4
/> : b 11 1 do i . loop ;
/> b
1 2 3 4 5 6 7 8 9 10
/> : c 10 0 do i . 2 +loop ;
/> c
0 2 4 6 8
/> : d 3 0 do ." Hip! " loop ." Hooray!" ;
/> d
Hip! Hip! Hip! Hooray!
```

**5.**

```
/> : stars ( n -- ) 0 ?do [char] * emit loop cr ;
/> : triangle ( n -- ) 1+ 1 do i stars loop ;
/> 4 triangle
*
**
***
****
```

**6.** `!` stores a number at an address; `@` fetches the number that's there; `+!` adds to it. Typing `score` alone leaves the variable's *address* on the stack: a number saying where in memory it is.

```
/> variable score
/> 0 score !
/> : hit ( -- ) 10 score +! ;
/> hit hit hit
/> score @ .
30
```

**7.** Yes. The file is on the SD card, and `include` reads it as if it were typed again.

**Questions.**

1. "Then" as in "and then, afterwards": it marks where the two paths join up again. Everything after `then` is done whichever way the `if` went.
2. The loop stops *before* its limit: `10 0 do` runs for 0 to 9, ten times. `0 0 do` runs 65,536 times, all the way round. `?do` skips the loop when the limit and the start are equal.
3. `do` only works while a definition is being compiled. It has to be inside a `: … ;`.
4. A variable's name gives its address, and it's read and written with `@` and `!`. A constant's name gives its value, which never changes. A value's name gives its value too, and `to` changes it.
5. Each small word can be tested alone (`6 say`), so when the whole thing is wrong, you know which part to suspect. And each has a name that says what it does, so the last word reads almost like a sentence.

**Challenge.** One way:

```
/> : divides? ( a b -- flag ) mod 0= ;
/> : prime? ( n -- flag ) dup 2 ?do dup i divides? if drop false unloop exit then loop
drop true ;
/> : primes ( n -- ) 1+ 2 do i prime? if i . then loop cr ;
/> 50 primes
2 3 5 7 11 13 17 19 23 29 31 37 41 43 47
```

Trying every divisor is slow: `1000 primes` takes a while, and beyond a few thousand it's a wait. Stopping at the square root of *n*, and skipping the even numbers, each help a great deal.

## Lab 18. hylang: Expressions and Functions

**1.**

```
hylang> (+ 1 2 3 4)
=> 10
hylang> (* 2 (+ 3 4))
=> 14
hylang> (- 10 (* 2 3))
=> 4
hylang> (/ 10 4)
=> 5/2
hylang> (> 5 3)
=> T
hylang> (> 3 5)
=> NIL
hylang> (+ "Hydra" "-" 16)
=> "Hydra-16"
```

**2.**

```
hylang> (* (+ 4 5) 2)
=> 18
hylang> (- 100 (* 7 8))
=> 44
hylang> (/ (+ 12 4) (- 9 5))
=> 4
hylang> (+ 1/2 1/3 1/6)
=> 1
```

**3.** In `def`, the name mustn't be worked out: `def` wants the *name itself*, not its value, and braces mean "leave this as it is". Afterwards, when the name is used, its value is exactly what's wanted, so it's written bare.

```
hylang> (def {age} 12)
=> NIL
hylang> (* age 365)
=> 4380
hylang> (def {name} "Sam")
=> NIL
hylang> (print "Hello," name)
Hello, Sam
=> NIL
```

**4.** The name is `square`. Its formal is `n`. Its body is `{* n n}`.

```
hylang> (fun {square n} {* n n})
=> NIL
hylang> (square 9)
=> 81
hylang> (square (square 3))
=> 81
```

**5.** For example:

```
hylang> (fun {double n} {* n 2})
=> NIL
hylang> (fun {cube n} {* n n n})
=> NIL
hylang> (fun {average a b} {/ (+ a b) 2})
=> NIL
hylang> (fun {greet who} {print (+ "Hello, " who)})
=> NIL
hylang> [(double 21) (cube 3) (average 3 4)]
=> {42 27 7/2}
hylang> (greet "Sam")
Hello, Sam
=> NIL
```

**6.**

```
hylang> (squre 5)
=> Error: Unbound Symbol 'squre'
hylang> (square 1 2)
=> Error: The function takes 1 argument, not 2
hylang> (square)
=> <function>(fn {n} {* n n})
hylang> (+ 1 "a")
=> Error: '+' adds numbers, or joins values to a string
```

**7.** hylang shows a prompt of spaces, and waits. Typing the missing `)` finishes the expression.

**Questions.**

1. Take any number of arguments: `(+ 1 2 3 4)`. With the sign in the middle, as at school, it can only ever have two.
2. True, and false. `NIL` also means "nothing", and the empty list.
3. `{* n n}` is a list of three things, kept as it is: code for later. `(* n n)` is a call, worked out at once. A function's body is in braces so that it's worked out when the function is *called*, not when it's defined.
4. `(to-fixed (/ 10 4) 1)` gives 2.5, and `(truncate (/ 10 4))` gives 2.
5. Making a new function out of an old one by filling in some of its arguments now, and the rest later:

```
hylang> (fun {add a b} {+ a b})
=> NIL
hylang> (def {add5} (add 5))
=> NIL
hylang> (add5 10)
=> 15
```

**Challenge.** hylang keeps the exact fraction, where HyForth could only give 36:

```
hylang> (fun {celsius f} {/ (* (- f 32) 5) 9})
=> NIL
hylang> (celsius 98)
=> 110/3
hylang> (to-fixed (celsius 98) 1)
=> 36.6
hylang> (fun {hypotenuse a b} {sqrt (+ (* a a) (* b b))})
=> NIL
hylang> (hypotenuse 3 4)
=> 5
hylang> (hypotenuse 1 1)
=> 1.41421356237
```

## Lab 19. hylang: Lists, Decisions and Programs

**1.**

```
hylang> (def {l} {5 3 8 1})
=> NIL
hylang> (len l)
=> 4
hylang> (fst l)
=> 5
hylang> (tail l)
=> {3 8 1}
hylang> (sort l)
=> {1 3 5 8}
hylang> (reverse l)
=> {1 8 3 5}
hylang> (sum l)
=> 17
hylang> (range 5)
=> {0 1 2 3 4}
hylang> (range 1 11)
=> {1 2 3 4 5 6 7 8 9 10}
```

**2.**

```
hylang> (map (fn {n} {* n 2}) {1 2 3})
=> {2 4 6}
hylang> (filter (fn {n} {> n 2}) {1 2 3 4})
=> {3 4}
hylang> (sum (map (fn {n} {* n n}) (range 1 11)))
=> 385
```

**3.** For example:

```
hylang> (sum (range 1 101))
=> 5050
hylang> (map (fn {n} {* n 2}) (range 1 11))
=> {2 4 6 8 10 12 14 16 18 20}
```

(`(range 2 21 2)` and `(filter even? (range 2 21))` give the same list.)

**4.**

```
hylang> (fun {size n} {if (> n 100) "big" "small"})
=> NIL
hylang> (map size {5 500 50 5000})
=> {"small" "big" "small" "big"}
```

**5.**

```
hylang> (dotimes {i 3} (print "Hip!"))
Hip!
Hip!
Hip!
=> NIL
hylang> (each {x {1 2 3}} (print (* x x)))
1
4
9
=> NIL
hylang> (fun {fact n} {if (< n 2) 1 (* n (fact (- n 1)))})
=> NIL
hylang> (fact 5)
=> 120
hylang> (fact 30)
=> 265252859812191058636308480000000
```

**6.** The hash's three names. `:name` is an **atom**: a name that stands for itself, used here as a key.

```
hylang> (def {pet} #({{:name "Rex"} {:legs 4}}))
=> NIL
hylang> (pet :name)
=> "Rex"
hylang> (hash-put pet {:colour "brown"})
=> NIL
hylang> (hash-keys pet)
=> {:name :legs :colour}
```

**7.** For example, in `lab19.hl`:

```
; lab19.hl - sizes, and a factorial
(fun {size n} {if (> n 100) "big" "small"})
(fun {fact n} {if (< n 2) 1 (* n (fact (- n 1)))})
(print "10 factorial is" (fact 10) "which is" (size (fact 10)))
```

### Questions.

1. `{+ 1 2}` is a list of three items, left alone. `(+ 1 2)` is a call, worth 3. `eval` takes a list and runs it as code: `(eval {+ 1 2})` is 3. So a program can build a piece of program, and then run it.
2. HyForth's numbers are 16-bit cells: nothing bigger than 65,535. hylang's whole numbers can be any size, and 30 factorial has 33 digits.

3. Each call is for a smaller number, and when the number is below 2 the `if` gives 1 without calling `fact` again. Every recursive function needs a case like that, and every call must move towards it.
4. The loop: going through the list, calling the function on each item, and collecting the answers.
5. A value can be used again: printed, put in a list, tested, passed to `map`. Something that's only been printed is gone. And a function that prints can't be tested by a program, only by eye.

**Challenge.** One way:

```
hylang> (fun {prime? n} {and (> n 1) (not (any? (fn {d} {== 0 (mod n d)}) (range 2
n)))})
=> NIL
hylang> (fun {primes n} {filter prime? (range 2 (+ n 1))})
=> NIL
hylang> (primes 30)
=> {2 3 5 7 11 13 17 19 23 29}
```

It's short, and it's slow: `(primes 100)` takes a good while, much longer than HyForth's. hylang's numbers can be any size, and that costs time on every sum (the manual's section 25.6).

## Lab 20. HyBASIC: First Programs

**1.**

```
> PRINT 2 + 3 * 4
 14
> PRINT (2 + 3) * 4
 20
> PRINT 10 / 4
 5/2
> PRINT 10 \ 4
 2
> PRINT 10 MOD 4
 2
> PRINT 2 ^ 10
 1024
> PRINT "Hy" + "dra"
Hydra
```

**2.** `;` puts the next item straight after the last (a number brings its own spaces); `,` moves to the next column, 14 characters on. `AGE = AGE + 1` means: work out `AGE + 1`, and put the answer in `AGE`. It's a command, "becomes", not a statement that two things are equal.

```
> AGE = 12
> NAME$ = "Sam"
> PRINT NAME$; " is"; AGE
Sam is 12
> PRINT NAME$, AGE
Sam            12
> AGE = AGE + 1
> PRINT AGE
 13
```

**3.** Typing a line again replaces it, and a new number adds a line:

```
> 10 FOR I = 1 TO 5
> 20 PRINT I; "times 7 is"; I * 7
> 30 NEXT I
> 10 FOR I = 1 TO 12
> 5 PRINT "The seven times table"
> LIST
5 PRINT "The seven times table"
10 FOR I = 1 TO 12
20 PRINT I; "times 7 is"; I * 7
30 NEXT I
```

**4.** `40 PRINT "Done."`, and something like `25 PRINT "-----"`. Any number from 21 to 29 will do. Numbering in tens leaves room for nine new lines between any two.

**5.** The first `LIST` shows nothing: `NEW` threw the program away. The second shows it all, back from the file.

**6.** HyBASIC says `?Redo from start`, and asks again: `A` is a number variable, and a word isn't a number.

**7.** For example:

```
30 IF A < 13 THEN
32   PRINT "Hello, young "; N$
34 ELSEIF A < 20 THEN
36   PRINT "Hello, "; N$
38 ELSE
39   PRINT "Good day, "; N$
40 END IF
45 PRINT "Next year you'll be"; A + 1
```

**Questions.**

1. With a number, the line is kept as part of the program, to be done when the program is `RUN`. Without, it's done at once, and not kept.
2. That the variable holds a string (text), not a number.
3. It's 0 (or an empty string). So a mistyped name isn't reported: it's quietly a *new* variable, worth 0, and the program runs on with the wrong answer.
4. `INPUT`, then the sum, then `PRINT`. BASIC does a program's lines in order, from the top, one at a time: a value has to be fetched before it's used, and worked out before it's printed.

5. `NEW` throws the program away. `LIST` shows it. `RUN` runs it. `SAVE` keeps it in a file, and `LOAD` brings it back from one.

**Challenge.** One way, as a file:

```
DO
  INPUT "Which table"; T
  FOR I = 1 TO 12
    PRINT USING "###"; I;
    PRINT " times"; T; "is";
    PRINT USING "#####"; I * T
  NEXT I
  INPUT "Another (y or n)"; A$
LOOP WHILE A$ = "y"
```

## Lab 21. HyBASIC: Bigger Programs

**1.** A `FOR` loop needs to know how many times to go round, and here that's the very thing the program is finding out. A `DO` loop goes on until its test says stop.

```
/ram> basic double.bas
 20 doublings make 1048576
```

**2.**

```
> A$ = "Hydra-16"
> PRINT LEN(A$)
 8
> PRINT LEFT$(A$, 5)
Hydra
> PRINT RIGHT$(A$, 2)
16
> PRINT MID$(A$, 2, 3)
ydr
> PRINT UCASE$(A$)
HYDRA-16
> PRINT INSTR(A$, "-")
 6
```

**3.** One way:

```
> 10 INPUT "Word"; W$
> 20 FOR I = LEN(W$) TO 1 STEP -1
> 30   PRINT MID$(W$, I, 1);
> 40 NEXT I
> 50 PRINT
> RUN
Word? Hydra
ardyH
```

**4.** `38/5`: exact, where a calculator would say 7.6. For a decimal, change the `PRINT` to `PRINT "The average is"; FIXED(TOTAL / 5, 2)`.

```
/ram> basic marks.bas
The average is 38/5
```

**5.** For example, before the `DATA` line:

```
HIGH = MARK(1) : LOW = MARK(1)
FOR I = 2 TO 5
  IF MARK(I) > HIGH THEN HIGH = MARK(I)
  IF MARK(I) < LOW THEN LOW = MARK(I)
NEXT I
PRINT "The highest is"; HIGH; "and the lowest is"; LOW
```

**6.** `ROWS` and `COLS` are the `SUB`'s **parameters**: its own names for the two values it's given. In the first call they're 3 and 5; in the second, 2 and 10.

```
/ram> basic box.bas
*****
*****
*****

**********
**********
```

**7.** Add the function, and a `PRINT` after each `BOX`:

```
BOX 3, 5
PRINT "Area:"; AREA(3, 5)

FUNCTION AREA (ROWS, COLS)
  AREA = ROWS * COLS
END FUNCTION
```

**Questions.**

1. `FOR`: when the number of times is known before the loop starts. `DO WHILE`: when it isn't, and the loop might need to run no times at all. `DO` ... `LOOP UNTIL`: when it isn't, and the loop must run at least once (asking a question until the answer is acceptable).
2. Six: `MARK(0)` to `MARK(5)`.
3. A `FUNCTION` gives back a value, and is used inside an expression: `PRINT AREA(3, 5)`. A `SUB` does something, and is used as a statement: `BOX 3, 5`.
4. Two procedures can each use `I`, or `TOTAL`, without spoiling the other's. Whoever writes one doesn't need to know the names used in the rest of the program.
5. So that people can see at a glance which lines belong to which loop or procedure. HyBASIC doesn't care.

**Challenge.** Project 1 is this challenge, grown up: Topic 15.

# Lab 22. Your Language Meets the Hydra

There are many right answers to each job. These are one for each language, the first three and the sixth as the Hydra ran them.

**In HyForth.**

```
/> s" date" sh-out type
2000-01-01 00:00:04
/> s" /ram/log.txt" w/o create-file throw constant f
/> s" one" f write-line throw s" two" f write-line throw s" three" f write-line throw
/> f close-file throw
/> variable g
/> : show ( -- ) 1 begin pad 80 g @ read-line throw while over . pad swap type cr 1+
repeat 2drop ;
/> s" /ram/log.txt" r/o open-file throw g !
/> show
1 one
2 two
3 three
/> g @ close-file throw
/> lib facility
/> : ready ( -- ) ." Ready..." cr 2000 ms ." Go!" cr ;
/> ready
Ready...
Go!
```

```
lib facility
: name ( -- ) page 34 12 at-xy yellow color ." Sam" plain cr ;

lib sound
: ceg ( -- ) 0 0 snd-patch  0 60 snd-note 400 ms  0 64 snd-note 400 ms  0 67 snd-note
400 ms  0 snd-off ;

lib video
320 8 bitmap  12 pen  60 40 260 200 box  160 120 60 circle
```

**In hylang.**

```
hylang> (sh-out "date")
=> "2000-01-01 00:00:05\n"
hylang> (write-file "/ram/log.txt" "one\ntwo\nthree")
=> NIL
hylang> (def {n} 0)
=> NIL
hylang> (each {l (read-lines "/ram/log.txt")} (do (set! {n} (+ n 1)) (print n l)))
1 one
2 two
3 three
=> NIL
hylang> (do (print "Ready...") (sleep 2) (print "Go!"))
Ready...
Go!
=> NIL
```

```
(use "screen")
(do (cls) (at 34 12) (color :yellow) (print "Sam") (plain))

(use "snd")
(tune {{"C4" 1} {"E4" 1} {"G4" 1}})

(use "video")
(do (bitmap 320 8) (pen 12) (box 60 40 260 200) (circle 160 120 60))
```

**In HyBASIC.**

```
> SHELL "date"
2000-01-01 00:00:05
> OPEN "/ram/log.txt" FOR OUTPUT AS #1
> PRINT #1, "one" : PRINT #1, "two" : PRINT #1, "three"
> CLOSE #1
> 10 OPEN "/ram/log.txt" FOR INPUT AS #1
> 20 N = 0
> 30 DO UNTIL EOF(1)
> 40   LINE INPUT #1, L$
> 50   N = N + 1
> 60   PRINT N; L$
> 70 LOOP
> 80 CLOSE #1
> 90 PRINT "Ready..." : SLEEP 2 : PRINT "Go!"
> RUN
 1 one
 2 two
 3 three
Ready...
Go!
```

```
CLS : LOCATE 12, 34 : COLOR 14 : PRINT "Sam"

PLAY "o4 l4 c e g"

SCREEN 13 : LINE (60, 40)-(260, 200), 12, B : CIRCLE (160, 120), 60, 14
```

**Questions.**

1. The student's own view, with a reason. Typically: HyBASIC has a statement for nearly everything; hylang has the shortest lines for files and lists; HyForth makes you build the most yourself.
2. HyForth needs a library for nearly everything here (`facility`, `sound`, `video`, loaded with `lib`); hylang for the screen, the sound and the video (loaded with `use`); HyBASIC for nothing: it's all built in.
3. Yes: all three can open `/dev/sndctl` or `/dev/vid/draw` and write the same lines of text that `echo` sent. The libraries are a convenience. Underneath, they do exactly that.
4. Each language reports it in its own way, and each lets a program catch it: HyForth's file words leave an error code (and `throw` and `catch` deal with it); hylang gives an error value, and `try` catches it; HyBASIC stops with a message, unless `ON ERROR GOTO` is set.

# Lab 23. Three Languages

**2.** Guess the middle of what's left, every time: 50, then 25 or 75, and so on. Each guess halves the possibilities, and 100 halved seven times is less than 1 (2 to the power 7 is 128, which is more than 100). So seven guesses are always enough.

**3.** In HyBASIC, for example: change the loop's last line, and add after it:

```
LOOP UNTIL GUESS = SECRET OR TRIES = 7
IF GUESS = SECRET THEN
  PRINT "Right! You took"; TRIES; "guesses."
ELSE
  PRINT "No more guesses. It was"; SECRET
END IF
```

**4.** From the manual's section 25.5, whichever two the student didn't type.

**5 and 6.** On the manual's Hydra, counting to 10,000 takes HyForth 40 ticks (a fifth of a second), hylang about 2.4 seconds, and HyBASIC about 2.7. Counting to 40,000 takes each about four times as long. The surprise is HyForth's answer:

```
/> : count-up ( n -- count ) 0 swap 0 do 1+ loop ;
/> 40000 count-up .
-25536
/> 40000 count-up u.
40000
```

40,000 doesn't fit in a signed 16-bit cell, whose largest value is 32,767. The count is right, bit for bit, but `.` reads the top bit as a minus sign. `u.` prints the same cell as an unsigned number.

**Questions.**

1. `play` calls itself: when the guess is wrong, the function's last act is to call `play` again. That's recursion, doing a loop's job.
2. `INPUT` prints the question, reads the line, and turns it into a number, all in one statement. In hylang, `read-line` reads and `val` converts. In HyForth, `accept` reads the characters, and `>number` converts them, and the program has to manage the buffer and the stack between the two.
3. HyForth's numbers are 16-bit cells, which the 65C02 adds in a few instructions. hylang and HyBASIC use numbers of any size, checked and handled by a library, at every sum. What's given up: big numbers, fractions and decimals (without a library), and a good deal of convenience.
4. That for most programs, the hours a person spends writing and correcting them matter far more than the seconds the computer spends running them. Choose the language that gets you a *correct* program soonest, and look for speed only where it's really needed.
5. A quiz: HyBASIC (strings, `INPUT` and `DATA` make it easy, and anyone can read it). The 200th Fibonacci number: hylang or HyBASIC (the number has 42 digits, far beyond HyForth's cells). A light a thousand times a second: HyForth, or machine code: the others can't get round a loop that fast.

**Challenge.** One way, in HyBASIC:

```
> 10 LO = 1 : HI = 100
> 20 DO
> 30   G = (LO + HI) \ 2
> 40   PRINT "Is it"; G
> 50   INPUT "s, b or r"; A$
> 60   IF A$ = "s" THEN LO = G + 1
> 70   IF A$ = "b" THEN HI = G - 1
> 80 LOOP UNTIL A$ = "r"
> 90 PRINT "I knew it."
> RUN
Is it 50
s, b or r? s
Is it 75
s, b or r? b
Is it 62
s, b or r? s
Is it 68
s, b or r? r
I knew it.
```

TOPIC 14

# Labs 24 to 26

## Lab 24. The 65C02, on Paper

**1.**

| REGISTER | BITS | WHAT IT'S FOR |
|---|---|---|
| A | 8 | The accumulator: arithmetic and logic are done here |
| X | 8 | An index register: counting, and pointing into tables |
| Y | 8 | Another index register |
| S | 8 | The stack pointer: where the next byte pushed on the stack goes |
| PC | 16 | The program counter: the address of the next instruction |
| P | 8 | The processor status: the flags |

**2.**

| THE SUM | A | CARRY |
|---|---|---|
| `$10` + `$20` | `$30` | 0 |
| `$0F` + `$01` | `$10` | 0 |
| `$80` + `$80` | `$00` | 1 |
| `$FF` + `$01` | `$00` | 1 |
| `$F0` + `$20` | `$10` | 1 |

A sum that passes `$FF` keeps only its low two hex digits in A, and the "one to carry" goes into the carry flag.

**3.**

| THE INSTRUCTION | A | X | Z | WHAT IT DID |
|---|---|---|---|---|
| `lda #$03` | `$03` | (unknown) | 0 | Put the number 3 in A |
| `tax` | `$03` | `$03` | 0 | Copied A into X |

| THE INSTRUCTION | A | X | Z | WHAT IT DID |
|---|---|---|---|---|
| `clc` | `$03` | `$03` | 0 | Cleared the carry, ready to add |
| `adc #$04` | `$07` | `$03` | 0 | Added 4 (and the carry, 0) to A |
| `dex` | `$07` | `$02` | 0 | Took 1 from X |
| `dex` | `$07` | `$01` | 0 | And again |
| `dex` | `$07` | `$00` | 1 | And again: the result is zero, so Z is set |

(Strictly, `clc` leaves Z as it was. A student who writes "unchanged" for flags that an instruction doesn't touch has understood more than one who writes 0.)

**4.** Four times: X counts 4, 3, 2, 1, and `bne` falls through when `dex` makes it 0. A is `$14` at the end, which is 20.

**5.** 4 × 5, by adding 5 four times. For 7 × 6: `ldx #$06` and `adc #$07` (or the other way round).

The Hydra will check both, and the challenge's sum too. HyBASIC can hold the routines, and show A afterwards:

```
> 10 T = 0
> 20 ASM
> 30 times:  lda #$00
> 40         ldx #$04
> 50 loop:   clc
> 60         adc #$05
> 70         dex
> 80         bne loop
> 90         rts
> 100 sum:   lda #$00
> 110        ldx #$0A
> 120 next:  stx t + 1
> 130        clc
> 140        adc t + 1
> 150        dex
> 160        bne next
> 170        rts
> 180 END ASM
> 190 CALL ASM times : RREG A, X, Y, P : PRINT A; HEX$(A)
> 200 CALL ASM sum : RREG A, X, Y, P : PRINT A; HEX$(A)
> RUN
 20 14
 55 37
```

**6.**

| THE INSTRUCTION | ITS MODE | THE BYTE IT LOADS IS |
|---|---|---|
| `lda #$41` | Immediate | The number `$41` itself |
| `lda $22` | Zero page | The byte at address `$0022` |
| `lda $1234` | Absolute | The byte at address `$1234` |

| THE INSTRUCTION | ITS MODE | THE BYTE IT LOADS IS |
|---|---|---|
| `lda $1234,x` | Absolute, X | The byte at `$1234` plus whatever is in X |
| `lda ($22),y` | Indirect, Y | The byte at the address held in `$22` and `$23`, plus Y |

**7.** `A9 41`; `85 22`; `60`.

**Questions.**

1. Machine code is the bytes the processor actually reads and obeys. Assembly language is the same instructions written as names that people can read; an assembler turns each line into its bytes.
2. It holds the address of the next instruction, and normally just moves on past each one. Jumps (`jmp`), branches, subroutine calls and returns (`jsr`, `rts`), and interrupts change it.
3. `lda #$41` puts the number `$41` in A. `lda $41` puts in A whatever byte is stored *at address* `$41`.
4. Up to 128 bytes back or 127 forward. Further than that takes a `jmp`.
5. The first 256 bytes of memory, `$0000` to `$00FF`. Instructions that use it are a byte shorter and a cycle quicker; and pointers (addresses used through `($22),y`) can *only* be kept there.
6. By adding, over and over, in a loop. Anything a bigger processor does in one instruction, a 65C02 does with a few simple ones.

**Challenge.** The second routine in the transcript above: A starts at 0; X counts down from 10; each time round, X is put into memory and added to A. The answer is 55, or `$37`. Adding up to 30 would give 465, which doesn't fit in a byte: A would hold only the low part, and the carry flag would be set by the addition that overflowed. So the program can find out by testing the carry after each `adc`, with `bcs`.

## Lab 25. Your First Machine Code

**1.** Nothing. From `as`, silence means success: it made the program `stars`.

**2.** For example:

| FIND | THE LINE |
|---|---|
| A comment | `; stars - a triangle of stars` |
| A constant being made | `ROWS = 5` |
| A variable being made | `row: .res 1` |
| A label | `main:` (or `line:`, `@row:`, `@star:`) |
| A system call | `jsr PUTC` |
| The instruction that ends the program | The `rts` at the end of `main` |

**3.** One line each time. With a name, the number is in one place, with a word that says what it means; if the 5 were written wherever it's needed, changing it would mean finding every one (and `ROWS + 1` would be a mysterious 6).

**4.**

| THE MISTAKE | WHAT AS SAID |
|---|---|
| `lad` for `lda` | `as: stars.s:15: not an instruction, directive or macro: lad` |
| `lda 1` for `lda #1` | Nothing: it's a valid instruction |
| `jsr lime` | `as: stars.s:18: undefined: lime` |
| No `.include "hydra.inc"` | `undefined: PUTC`, twice, and two more for names the header needs |

Each message has the file's name and the line's number. The second mistake is the important one. `lda 1` loads the byte *at address 1*, which is the ROM bank register, and so the triangle starts with a row as long as whatever number happens to be there, and carries on from that. The assembler can only check that a program is *written* correctly, never that it does what was meant.

**5.**

```
/ram> as -l stars.s
/ram> cat stars.lbl
al 000023 .count
al 000830 .main
al 000022 .row
al 000842 .line
```

`main` is at `$0830`, and `line` at `$0842`. A program is loaded at `$0800`, but its first 48 bytes (`$30`) are its header, so the code begins at `$0830`.

**6.** The program is 83 bytes, and its source about ten times that. The source has what people need and the processor doesn't: the comments, the names, the layout.

```
/ram> ls -l stars stars.s
--rw-rw-rw- fr       83 2000-01-01 00:01 stars
--rw-rw-rw- fr      873 2000-01-01 00:00 stars.s
```

**7.** One way: start `row` at `ROWS`, count down, and stop at 0.

```
main:       lda #ROWS           ; The longest row first
            sta row
@row:       lda row
            jsr line
            dec row             ; The next row is one shorter, till none is left
            bne @row
            rts
```

**Questions.**

1. It turns a text file of assembly language (the source) into a file of machine code that the Hydra can run (the program).
2. A system call may change A, X and Y. `line` calls `PUTC` for every star, so a count kept in X wouldn't survive. Memory does.

3. `.zeropage`, before their lines. `row` is at `$22` and `count` at `$23`: the first two bytes of the program's own part of the zero page.
4. A request to HydraOS, made by `jsr` to a fixed address that has a name. `stars` uses `PUTC`, which prints the character whose code is in A.
5. `lda 1` reads memory address 1, and whatever is there, when a programmer nearly always means the number 1. It's the commonest bug in 6502 programming, and the assembler can't warn about it.

**Challenge.** One way to print 0 to 9:

```
.include "hydra.inc"
.include "hyx2.inc"
.include "macros.inc"

            HYX2_PROGRAM "count", main

.zeropage
digit:      .res 1

.code
main:       stz digit           ; From 0
@next:      lda digit
            clc
            adc #'0'            ; A digit's code is $30 plus the digit
            jsr PUTC
            inc digit
            lda digit
            cmp #10
            bne @next
            lda #LF
            jsr PUTC
            rts
```

## Lab 26. Taking Programs Apart

**1.** Without the labels file, `dis` makes up names for the places that are jumped to (`L0834`) and shows variables as bare addresses (`$22`). With it, the program's own names come back: `row`, `count`, `line`. What never comes back is everything that wasn't in the program to begin with: the comments, the constant `ROWS` (it's just a 6 now), the layout.

```
/ram> dis stars | head -12
; stars, as dis read it: as assembles this to its bytes again
.include "hydra.inc"

.include "hyx2.inc"

        HYX2_PROGRAM "stars", main
.code
main:
        lda #$01
        sta $22
L0834:
        lda $22
/ram> dis -l stars.lbl stars | head -12
; stars, as dis read it: as assembles this to its bytes again
.include "hydra.inc"
row             = $22
count           = $23

.include "hyx2.inc"

        HYX2_PROGRAM "stars", main
.code
main:
        lda #$01
        sta row
```

**2.** `cmp` says nothing, which means the two files are the same, byte for byte: so `echo` prints `the same`. It proves that the disassembly is complete and exact: assembling it gives back the very same program.

```
/ram> dis stars >stars2.s
/ram> as stars2.s stars2
/ram> cmp stars stars2 && echo the same
the same
```

**3.** Any of the calls it makes: a `jsr` to a name from `hydra.inc`, such as `jsr WRITE`.

**4.** `db` shows the task it started the program in, the registers, and the next instruction. The program counter is `$0830`, the program's first instruction. A holds whatever was left in it: `stars` hasn't run a single instruction yet.

```
/ram> db ./stars
task 3
PC=0830 A=30 X=FF Y=00 S=FD P=nv--dizc W=0 U=0 RAM=00 ROM=00
0830  A9 01     lda #$01
db> l /lib/as/hydra.inc
108 symbols
db> l stars.lbl
4 symbols
db> d main 9
0830  A9 01     lda #$01                ; main
0832  85 22     sta row                 ; main+2
0834  A5 22     lda row                 ; main+4
0836  20 42 08  jsr line                ; main+6
0839  E6 22     inc row                 ; main+9
083B  A5 22     lda row                 ; main+B
083D  C9 06     cmp #$06                ; main+D
083F  D0 F3     bne main+4              ; main+F
0841  60        rts                     ; main+11
```

**5.**

```
db> s
PC=0832 A=01 X=FF Y=00 S=FD P=nv--dizc W=0 U=0 RAM=00 ROM=00
0832  85 22     sta row                 ; main+2
db> s
PC=0834 A=01 X=FF Y=00 S=FD P=nv--dizc W=0 U=0 RAM=00 ROM=00
0834  A5 22     lda row                 ; main+4
db> s
PC=0836 A=01 X=FF Y=00 S=FD P=nv--dizc W=0 U=0 RAM=00 ROM=00
0836  20 42 08  jsr line                ; main+6
db> s
PC=0842 A=01 X=FF Y=00 S=FB P=nv--dizc W=0 U=0 RAM=00 ROM=00
0842  85 23     sta count               ; line
db> s
PC=0844 A=01 X=FF Y=00 S=FB P=nv--dizc W=0 U=0 RAM=00 ROM=00
0844  A9 2A     lda #$2A                ; line+2
```

| I TYPED | PC | A | THE NEXT INSTRUCTION | WHAT JUST HAPPENED |
|---|---|---|---|---|
| `s` | 0832 | 01 | `sta row` | `lda #$01`: A became 1 |
| `s` | 0834 | 01 | `lda row` | `sta row`: the 1 went into `row` |
| `s` | 0836 | 01 | `jsr line` | `lda row`: A loaded from `row` (still 1) |
| `s` | 0842 | 01 | `sta count` | `jsr line`: the program counter jumped into `line`, and S went down by 2 |
| `s` | 0844 | 01 | `lda #$2A` | `sta count`: the 1 went into `count` |

**6.** After the first two steps, `row` is 1, as it should be. `count` holds whatever was in that byte of memory before: the program hasn't set it yet. Memory isn't zero until something makes it so.

**7.** Each `c` runs the program on to the breakpoint, at the instruction that's about to print a new line. So between stops it prints a row of stars (one more each time) and the new line from the time before. `r` shows the registers at the stop: PC is `$084D` each time, and A is `$2A`, the star it has just printed.

```
db> b line+B
db> c
*breakpoint 1
PC=084D A=2A X=FF Y=00 S=FB P=nv--diZc W=0 U=0 RAM=00 ROM=00
084D  A9 0A     lda #$0A                ; line+B
db> r
PC=084D A=2A X=FF Y=00 S=FB P=nv--diZc W=0 U=0 RAM=00 ROM=00
084D  A9 0A     lda #$0A                ; line+B
db> c

**breakpoint 1
PC=084D A=2A X=FF Y=00 S=FB P=nv--diZc W=0 U=0 RAM=00 ROM=00
084D  A9 0A     lda #$0A                ; line+B
```

**8.** The rest of the triangle is printed in `#`: the program in memory now has `$23` where it had `$2A`. Neither file has changed. `stars.s` still says `'*'`, and `stars` on the disk still has `$2A` in it: run it again, and the stars are back. `db` changed only the copy in the task's memory.

```
db> d line 3
0842  85 23     sta count               ; line
0844  A9 2A     lda #$2A                ; line+2
0846  20 50 F9  jsr PUTC                ; line+4
db> w 0845 23
db> c
#
##
###
####
#####
task 3 ended: code 0
```

**Questions.**

1. For programs whose source you don't have; for checking what the assembler really produced; and for learning how someone else's program works.
2. Nothing in a program's bytes says which are instructions and which are data: a string looks like any other bytes. `dis` follows the program's flow from its start, and what it can't reach as code, or what isn't a valid instruction, it shows as data (`.byte`).
3. `s` steps *into* the subroutine: the next stop is its first instruction. `n` runs the whole subroutine and stops at the instruction after the `jsr`.
4. A place where the program is to stop. `db` writes a `BRK` instruction over the instruction there (remembering what was underneath). When the processor reaches it, HydraOS stops the task and tells `db`.
5. Make the same change in the source, and assemble it again. The fix in memory is gone as soon as the program ends.

**Challenge.** The manual's section 29.8 is the answer. The bug: `LDR r0, msg` loads a register pair and, on the way, changes A, which held the error code that the next call needed. The fix is to save A on the stack round the `LDR`: `pha` before, `pla` after. A student who finds that A changes across a line that "shouldn't" change it has found it.

TOPIC 15

# The Projects

For each of the Companion's seven projects: the answers to "Understand it first", what a good finished project looks like, the snags to expect, and a worked solution to one or two of the extensions. Topic 7 has the rubric.

A general rule for all seven: **the starting listing is not the project.** A student who has typed it in and run it has done the equivalent of a lab. The project is what they do to it next.

## Project 1. A Quiz

**Understand it first.**

1. It's the number of questions: `READ COUNT` takes it, and the loop runs that many times. Add a question, and the 4 must become a 5, or the new one is never asked. (A better design, and a good extension: end the data with a marker, such as `"end"`, and loop until it's read.)
2. So that capitals don't matter: `65c02`, `65C02` and `65c02` typed with the shift key stuck all count.
3. `INPUT` treats a comma as the end of an answer, so "Paris, France" would be cut short. `LINE INPUT` takes the whole line, commas and all.

**Look for** questions the student cares about; messages that have a voice; and input that's been thought about (a blank answer, an answer with spaces round it: `LTRIM$` and `RTRIM$`).

**Expect** the `DATA` count to be wrong after questions are added (an error from `READ`, or a question that's never asked); and commas *inside* a question, which split it in two unless the question is in quotes.

**A worked extension**: an answer that's right if it's anywhere in the reply.

```
> A$ = "it is the 65c02, I think" : B$ = "65C02"
> IF INSTR(UCASE$(A$), UCASE$(B$)) > 0 THEN PRINT "Right!" ELSE PRINT "No."
Right!
```

In the program, that's one changed line: `IF INSTR(UCASE$(REPLY$), UCASE$(ANSWER$)) > 0 THEN`.

# Project 2. Turtle Pictures

**Understand it first.**

1. 120 degrees for a triangle; 60 for a hexagon. The rule: the turtle turns through a full circle, 360 degrees, on its way round any closed shape, so each corner is 360 divided by the number of sides. In `polygon`, that's `360 over /`.
2. For `60 4 polygon`:

| THE WORD | THE STACK AFTERWARDS |
|---|---|
| (to begin with) | 60 4 |
| `360` | 60 4 360 |
| `over` | 60 4 360 4 |
| `/` | 60 4 90 |
| `swap` | 60 90 4 |
| `0` | 60 90 4 0 |
| `do` | 60 90 (the loop has taken the 4 and the 0) |

3. It chooses the pen's colour from the loop's count: `i 15 mod` runs from 0 to 14 and round again, and `1+` makes that 1 to 15. Without the `1+`, one square in fifteen would be drawn in colour 0, which is black, and wouldn't be seen.

**Look for** words built on words (`flower` uses `square` uses `polygon`), a stack diagram on every one, and a picture that the student planned rather than stumbled on.

**Expect** stack trouble inside loops: a word that leaves a number behind each time round overflows the stack by the end of a flower. The cure is the stack diagram, checked against the word a step at a time, as in the table above.

**Worked extensions.**

```
: star ( size -- ) 5 0 do dup fd 144 rt loop drop ;
: spiral ( turn -- ) 100 1 do i 2 * fd dup rt loop drop ;
cs 80 star
cs 91 spiral
```

A five-pointed star turns 144 degrees at each point, because it goes round *twice* on its way: 720 divided by 5. That's the answer to the challenge's "why": a turn that divides evenly into 360 makes a polygon, one that divides into a multiple of 360 makes a star, and one that divides into neither never closes, and makes a spiral that slowly rotates.

# Project 3. A Round for Two Voices

**Understand it first.**

1. Line `B` starts with `r1 r1`, two whole-note rests, and it has another instrument. So the second voice comes in two bars after the first, with the same tune.
2. `l4`: notes are quarter notes unless they say otherwise. `g2`: a G for a half note. `g8`: a G for an eighth. `[c d e c]2`: play what's in the brackets twice. `<g>`: down an octave, a G, and up again. `r1`: a rest for a whole note, four beats.
3. Four eighths are two beats, and two quarters are two more: four beats. A whole bar.
4. 73 is the flute and 71 the clarinet (General MIDI's numbers, counted from 0: section 15.4).

**Look for** bars that add up, in every voice, all the way through; a score that's laid out so the bars can be seen (the `|` is there for that); and, in a tune of the student's own, some shape: a phrase that comes back.

**Expect** voices that drift apart, from a bar with too many or too few beats in it. Have the student count each bar aloud. `play -o` will turn a score into a ZSM file without playing it, which catches spelling mistakes in the MML (it reports the line) before a class of thirty all press play.

**A worked extension**: a third voice, two bars later still, and each voice in its own speaker.

```
A p l I73 o4 l4 [c d e c]2 [e f g2]2 [g8 a8 g8 f8 e c]2 [c <g> c2]2
B p r I71 o4 l4 r1 r1 [c d e c]2 [e f g2]2 [g8 a8 g8 f8 e c]2 [c <g> c2]2
C p c I0 o3 l4 r1 r1 r1 r1 [c d e c]2 [e f g2]2 [g8 a8 g8 f8 e c]2 [c <g> c2]2
```

## Project 4. Your Own Command

**Understand it first.**

1. `$*` is the list of the script's arguments, and `$#*` is how many there are: 0, when `note` is typed alone.
2. It runs `date`, and puts its output into the line as words: the date, and the time. `>>` adds the line to the end of the notebook. `>` would empty the notebook first, every time.
3. Into `/ram/bin`: new files in `/bin` go to the RAM disk, and are gone at the next reset. To keep the command, make the directory `/sd/0/bin` and put the script there. It joins `/bin` when the namespace is next built: at a reset, or straight away with the command `newns`.

**Look for** a first comment that says how to use the command; behaviour that's sensible when the notebook doesn't exist yet; and a usage message.

**Expect** the usage message to want quotes. In rc, `[` and `]` make a pattern, and `[-n]` would be replaced by the name of any file in the current directory that it matched (when nothing matches, rc leaves the word alone, so the mistake hides until it bites). Keep the message in quotes. It's a good moment to ask what else rc has been doing to their words all along.

**A worked solution**, with three options and a usage line:

```
#!/bin/rc
# note - a notebook.  note words ... adds a line; note alone shows them all;
# note -n counts them; note -l shows the last five; note -f word finds the ones with a
word in
book=/sd/0/work/notebook
if(~ $#* 0) {
        cat $book
        exit
}
switch($1){
case -n
        wc -l <$book
case -l
        tail -5 $book
case -f
        grep $2 $book
case -*
        echo 'usage: note [-n] [-l] [-f word] [words ...]'
case *
        echo `{date} $* >>$book
}
```

```
/ram> rc note2 -n
      2
/ram> rc note2 -f solder
2000-01-01 00:01:13 Buy solder
/ram> rc note2 -z
usage: note [-n] [-l] [-f word] [words ...]
```

## Project 5. A Program in the Background

**Understand it first.** The two lines of `ps` are an `rc`, running the script, in the state `wait`; and a `sleep`, its child, in the state `sleep`. The script spends nearly all its time waiting for its `sleep` to finish.

1. The kernel gives each task that's ready a short turn at the processor, two hundred times a second, and takes it away again at the next tick. The shell gets turns between the ticker's.
2. Almost none: a task that's asleep or waiting isn't given turns at all. The `cpu` column for both stays at 0.0.
3. CTRL-C at the prompt goes to the program in the foreground, and a command started with `&` isn't that: the ticker carries on. A reset stops everything, the ticker with it.

**Look for** a student who can start it, find it and stop it without help, and who can say what `&` does. The written explanation in the "two tasks, one file" step is the heart of it.

**Expect** forgotten tickers: three of them still running, a week later, in a student's long-lived emulator. `ps` at the start of each lesson is a good habit. And expect "which number do I kill?": the `rc`, not the `sleep` (kill the `sleep`, and the loop just starts the next one).

**Two tickers, one file.** In practice their lines don't get mixed: each `date` writes its whole line in one go, and the file is opened for adding to its end. That's the contrast with `chorus`, where each task writes a *letter* at a time. Whether tasks tread on each other depends on how big a step each one takes with the thing they share: which is the whole idea of a lock.

**The challenge** (`race`). Adding 1 takes three steps: read the counter, add 1, write it back. If one task is stopped after its read, and another task then reads the same value, both write back the same number, and two additions have counted as one. A lock makes the three steps into one: a task that holds it finishes its read, add and write before any other task can begin.

## Project 6. A Traffic Light

**Understand it first.**

1. `on 2` writes a 0 to `/dev/gpio/2`. The LEDs are wired from +5 V to the pins, so current flows, and an LED lights, when its pin is *low*.
2. It makes pins 2, 3 and 4 outputs, and switches all three LEDs off, so that the lights start from a known state whatever was there before.
3. Red, for 4 seconds; red and amber together, for 1; green, for 4; amber, for 1; and round again.

**Before any power goes on,** check each circuit yourself (Topic 4, and the lesson notes for Lab 14): a resistor for every LED, nothing on the header's pins 3 and 4, and no stray wire across +5 V and ground.

**Look for** a drawing that matches the wiring; a `lightsoff`; and, in the crossroads version, a student who has *argued* that the two roads can never both be green, and not just watched it for a while.

**Expect** an LED in backwards (it simply doesn't light: turn it round), and lights left on when the script is stopped with CTRL-C half way through.

**A worked extension**: `lightsoff`.

```
#!/bin/rc
# lightsoff - put the traffic light out
for(p in 2 3 4) echo 1 >/dev/gpio/$p
```

## Project 7. Faster, with Machine Code

**Understand it first.**

1. On the manual's Hydra, about fifty times: 2.7 seconds against a twentieth of a second.
2. With two registers together: X is the low byte of a 16-bit count, and Y the high byte.
3. `inx` adds 1 to X every time round. `iny` happens only when X goes round from `$FF` to `$00`: the 256th time.
4. 10,000 is `$2710`: the count has reached it when the low byte is `$10` and the high byte is `$27`. X is `$10` once in every 256 counts, so the routine must also check that Y is `$27`: only then is it the right `$10`.

5. It copies the registers, as the routine left them, into four HyBASIC variables. `Y * 256 + X` puts the two bytes back together: the high byte is worth 256 times as much.

**Look for** routines that have been run by hand, on paper, before they were typed; a comment on every line; and a student who can say what each register is for.

**Expect** crashes. A machine-code routine that never reaches its `rts`, or that writes where it shouldn't, can take HyBASIC down with it, and sometimes needs a reset. That's normal at this level, and it's why the work should be saved before every run. A loop that never ends is stopped with CTRL-C as usual.

**A worked extension**: the number to count to, passed in. The registers arrive already loaded, so the routine keeps the target in two bytes of a HyBASIC variable, and counts until it gets there.

```
T = 0
LIMIT = 50000
ASM
count:  stx t + 1       ; The target's low and high bytes, kept in the variable T
        sty t + 2
        ldx #0
        ldy #0
again:  inx
        bne skip
        iny
skip:   cpx t + 1
        bne again
        cpy t + 2
        bne again
        rts
END ASM
CALL ASM count, 0, LIMIT MOD 256, LIMIT \ 256
```

**The challenge.** A loop that runs once at start-up: HyBASIC, since nobody will notice the time. A loop over every dot of a picture (76,800 of them): machine code, or the wait is minutes. A program to run once and throw away: HyBASIC, every time. The time that matters is the programmer's.

TOPIC 16

# A Bank of Questions

Sixty short-answer questions, in six sets of ten, with answers. Each set goes with a stretch of the course, and runs from recall (the first few) to explanation (the last few). Use them as a starter, an exit ticket, a homework or a test: Topic 7 says how they fit.

The answers are models. Any answer that shows the same understanding is right.

## Set 1. The machine (Chapters 1 and 7; Labs 1 and 4)

| | QUESTION | ANSWER |
|---|---|---|
| 1 | What is the Hydra's processor called? | The W65C02S (a 65C02). |
| 2 | How many bits are in a byte, and how many values can a byte hold? | Eight; 256 (0 to 255). |
| 3 | Write 255 in hex, and `$10` in decimal. | `$FF`; 16. |
| 4 | What's the difference between RAM and ROM? | RAM can be written and forgets when the power goes off; ROM keeps its contents and can't be changed by a program. |
| 5 | How many tasks can the Hydra run at once, and how much memory of its own does each have? | Sixteen; 32K each (plus RAM banks from the memory cards). |
| 6 | What is the bus? | The paths that join the processor to memory and to the other chips: addresses, data and control lines. |
| 7 | The 65C02 can address 64K. How does the Hydra give it megabytes? | Parts of the 64K are "windows" onto bigger memories; a register chooses which piece (bank) shows through each. |
| 8 | What is the kernel, and where is it kept? | The heart of HydraOS, which runs the other programs and shares the machine between them; in the kernel ROM, U6. |
| 9 | What does the first program, `init`, do? | Sets the system up (the console, the RAM disks, the namespace), then becomes the shell. (The kernel starts `init` again if that shell ends, and for each new window.) |
| 10 | What stops one task from spoiling another's memory? | Each task's 32K is chosen by the T register, which only the kernel changes; a task has no address that reaches another task's RAM. |

## Set 2. Using HydraOS (Chapters 6 and 8 to 11; Labs 3 and 5 to 9)

| | QUESTION | ANSWER |
|---|---|---|
| 1 | What does the prompt `/>` tell you? | HyForth, the shell, is ready for a line; and the current directory is `/`. |
| 2 | Which key stops a running program? Which says "my typing has ended"? | CTRL-C; CTRL-D. |
| 3 | Why is `/ram` a poor place to keep work? Where is a good one? | It's memory, emptied at every reset and power-off. The SD card, `/sd/0`. |
| 4 | What's the difference between > and >>? | > sends output to a file, replacing what was there; >> adds it to the file's end. |
| 5 | What does `ls /rom/bin \| wc -l` do? | Counts the names in `/rom/bin`: `ls`'s output is piped into `wc -l`, which counts lines. |
| 6 | In rc, `x=(a b c)`. What are `$#x` and `$x(2)`? | 3; `b`. |
| 7 | How does the shell decide whether a line is HyForth's or rc's? | By its first word: a Forth word or a number makes it Forth; anything else goes to rc. |
| 8 | What's a path? Give a full one and a relative one. | The directories that lead to a file. `/sd/0/work/a.txt` starts from the root; `work/a.txt` from the current directory. |
| 9 | What does `sleep 60 &` do, and how would you stop it? | Starts `sleep` in the background, so the prompt comes straight back. Find its task with `ps`, and `kill` that number. |
| 10 | What's a script, and how do you make one into a command? | A file of commands. Start it with `#!/bin/rc`, and put it in a directory that's part of `/bin` (`/sd/0/bin`, to keep it). |

## Set 3. Devices (Chapters 12 to 17; Labs 10 to 14)

| | QUESTION | ANSWER |
|---|---|---|
| 1 | What is a driver? | A program that looks after a device, and offers it to other programs as files. |
| 2 | What are the two things you can do with a control file? | Read it, to see the device's settings; write a line to it, to give the device a command. |
| 3 | What does `invalid argument` mean, from a device? | The driver didn't understand the command, or a number was out of range. |
| 4 | Name the Hydra's three kinds of disk. Which keeps its files when the power is off? | SD cards, RAM disks and the ROM disk. The SD cards (and the ROM disk, which can't be changed). |
| 5 | What's a block? What's HydraFS? | 512 bytes: the unit a disk is read and written in. The Hydra's file system: its way of arranging files on a disk. |

| | QUESTION | ANSWER |
|---|---|---|
| 6 | What note is number 60? How much higher is note 72? | Middle C. An octave. |
| 7 | In MML, what do `o4`, `l8` and `r` mean? | Octave 4; notes are eighth notes unless they say otherwise; a rest. |
| 8 | On the Vera X's bitmap, where is the dot (0, 0)? Which way does *y* go? | The top left. Down. |
| 9 | Why does an LED on a pin need a resistor, and why is it wired from +5 V to the pin? | To limit the current. Because a pin can take in more current than it can give out, so the LED lights when the pin is 0. |
| 10 | "On the Hydra, everything is a file." What does that buy? | One set of commands and one set of file words reach everything: disks, devices, even the tasks. Nothing new to learn for each device. |

## Set 4. Numbers and programming (Chapters 18 to 25; Labs 15 to 23)

| | QUESTION | ANSWER |
|---|---|---|
| 1 | What does `calc '10 / 4'` print, and why isn't it 2.5? | `5/2`. The Hydra keeps fractions exactly, as fractions. |
| 2 | Write 10 in binary and in hex. | 1010; A. |
| 3 | In HyForth, what does `2 3 + 4 * .` print? | 20. |
| 4 | What do `dup`, `swap` and `drop` do? | Copy the top of the stack; exchange the top two; throw the top away. |
| 5 | In hylang, what's the value of `(* 2 (+ 3 4))`? | 14. |
| 6 | In hylang, what's the difference between `{+ 1 2}` and `(+ 1 2)`? | The first is a list, left as it is; the second is a call, worked out: 3. |
| 7 | In HyBASIC, what does `A = A + 1` mean? | Work out `A + 1`, and put the answer back in `A`: add one to `A`. |
| 8 | What's the difference between a `FOR` loop and a `DO` loop? | `FOR` counts a known number of times; `DO` goes on while (or until) a test is true. |
| 9 | A variable, a loop and a decision: write one line of each, in any one of the three languages. | For example, in HyBASIC: `N = 5`; `FOR I = 1 TO 10 : PRINT I : NEXT`; `IF N > 3 THEN PRINT "big"`. |
| 10 | HyForth counts to ten thousand more than ten times faster than HyBASIC. Why, and what does it cost? | Its numbers are 16-bit cells, which the processor adds directly; HyBASIC's can be any size, and are handled by a library. The cost: no big numbers or fractions without extra work. |

## Set 5. Machine language (Chapters 26 to 29; Labs 24 to 26)

| | QUESTION | ANSWER |
|---|---|---|
| 1 | Name the 65C02's three 8-bit registers for data. | A, X and Y. |
| 2 | What does the program counter hold? | The address of the next instruction. |
| 3 | What's the difference between `lda #$10` and `lda $10`? | The first loads the number `$10`; the second loads the byte stored at address `$10`. |
| 4 | `$FF` + `$01`, in one byte: what's the result, and what's the carry? | `$00`, with a carry of 1. |
| 5 | What does `bne loop` do? | Goes to `loop` if the last result wasn't zero (the Z flag is 0); otherwise carries on. |
| 6 | What do `jsr` and `rts` do? | Call a subroutine (remembering where to come back to, on the stack); return from one. |
| 7 | What's the zero page, and why is it special? | Addresses `$0000` to `$00FF`. Instructions that use it are shorter and quicker, and pointers must be kept there. |
| 8 | What does an assembler do? And a disassembler? | Turns assembly language into machine code; turns machine code back into assembly language. |
| 9 | In `db`, what do `s`, `b` and `c` do? | Step one instruction; set a breakpoint; continue to the next breakpoint or the end. |
| 10 | A program assembles with no errors, and prints the wrong thing. What does that tell you, and what do you do? | The assembler only checks that the program is written correctly, not that it's right. Step through it in `db`, comparing each register with what you expected. |

## Set 6. Building (Chapters 3 to 5; the build log)

| | QUESTION | ANSWER |
|---|---|---|
| 1 | Name three safety rules for soldering. | Any three: glasses on; the iron in its stand; don't breathe the fumes; no food or drink; wash your hands; unplug the power before touching the board. |
| 2 | What do you do for a burn? | Cool running water, at once, for at least ten minutes; tell an adult. |
| 3 | A resistor's bands are brown, black, orange. What's its value? | 10 kΩ (1, 0, and three zeros). |
| 4 | A capacitor is marked 104. What's its value? | 100 nF (10, and four zeros, in picofarads). |
| 5 | Which parts must go in the right way round? | Electrolytic capacitors, chips and their sockets, resistor networks, the reset chip, LEDs. |

| | QUESTION | ANSWER |
|---|---|---|
| 6 | What does a good solder joint look like? | A small, shiny cone that has flowed onto both the pad and the lead. |
| 7 | Why is the power tested before any chip goes in? | A power fault found then harms nothing; with the chips in, it could destroy them. |
| 8 | What are the terminal's settings for a Hydra? | 9600 baud, 8 data bits, no parity, 1 stop bit, flow control XON/XOFF. |
| 9 | The power light is on, and nothing appears on the terminal. Name the first three things to check. | The port and the speed; the cable (RS-232, not TTL; not null-modem); press reset. Then the jumpers and the chips. |
| 10 | Why change only one thing at a time when finding a fault? | So that you know which change made the difference. Change two things, and whatever happens, you've learned nothing. |